

第三届中国研究生操作系统开源创新大赛

暨开放原子大赛操作系统专项赛

第 16 题：云一边一端异构计算资源调度

基于深度强化学习的 云一边一端异构计算资源管理调度方法

项目说明书

参赛队伍：操作系统创新小分队

二〇二六年九月

目 录

摘要	1
阅读指南、编写依据与组织说明	3
第一章 赛题问题、数学模型与验收映射	5
1.1 问题定义	5
1.2 默认异构资源	5
1.3 评价指标	6
1.4 赛题要求到代码与证据的闭环	6
1.5 官方评分项逐项验收说明	7
1.6 建模假设与适用范围	8
1.7 问题难点与采用混合方法的原因	8
第二章 系统架构与调度物理规则	10
2.1 分层架构	10
2.2 插入式资源时间线	11
2.3 观测空间与归一化	11
2.4 Ranked 动作空间与 EFT 资源选择	12
2.5 相对 HEFT 奖励	12
2.6 单个 episode 的状态转移	12
2.7 调度器统一接口与返回契约	13
2.8 调度合法性的五层防线	13
2.9 公平比较的实现规则	14
第三章 技术演进：从诊断到残差调度	15
3.1 演进总览	15

3.2 策略塌缩的发现	16
3.3 归一化诊断	16
3.4 动作空间重构	16
3.5 残差式排序策略	17
3.6 Best-of-N 与精修层	17
3.7 未被采用的方向	18
3.8 与相关方法的关系	18
3.9 关键设计选择及其取舍	19
3.10 从失败实验得到的工程原则	19
第四章 最终方案与算法流程	20
4.1 完整流水线	20
4.2 正确性与不变量	20
4.3 算法定位	21
4.4 Residual 构造阶段伪代码	21
4.5 单任务重定位与 LNS 伪代码	22
4.6 自适应候选组合与宽并行修复	22
4.7 非回归、容量安全与时间预算	23
4.8 时间复杂度与资源开销分析	23
4.9 异常与降级路径	24
第五章 实验协议与统计方法	25
5.1 数据、训练与验证隔离	25
5.2 基线	25
5.3 重复与配对设计	25
5.4 场景生成、划分与分布外评测	26

5.5 指标定义与统计解释	26
5.6 实验控制与可复现措施	27
5.7 测试策略	27
第六章 结果与证据	28
6.1 最终方案的五次重复结果	28
6.2 计算量对齐验证	29
6.3 与 MILP 理论最优的距离	29
6.4 结构化泛化	30
6.5 大规模泛化与容量安全	31
6.6 openEuler 与测试验证	32
6.7 主结果、历史结果与大规模结果的统一解释	33
6.8 改进幅度与稳定性	33
6.9 负面证据与结论边界	34
第七章 复现、审计与工程交付	35
7.1 依赖与关键配置	35
7.2 一键运行	35
7.3 单次 LNS 调试命令	36
7.4 测试与报告审计	37
7.5 统一复现入口与三个档位	37
7.6 单场景真实调度接口	38
7.7 国产操作系统 Docker 矩阵	38
7.8 项目目录与数据流	39
7.9 依赖安装与无 GPU 运行	40
7.10 常见问题定位	40

第八章 局限性、适用边界与未来工作	42
8.1 已知局限	42
8.2 后续方向	42
8.3 内部有效性威胁	43
8.4 外部有效性威胁	43
8.5 构念与统计有效性	43
8.6 面向实际系统的部署方式	44
8.7 决赛前建议路线图	44
第九章 创新点与工程价值	45
9.1 方法创新	45
9.2 工程创新	45
9.3 实际价值	45
第十章 结论	47
参考文献	48
附录 A 关键证据索引	50
附录 B 关键代码索引	51
附录 C 匿名化与数据口径声明	52
附录 D 最终训练与推理配置	53
附录 E 单场景输入输出字段说明	54
附录 F 自动化测试覆盖矩阵	55
附录 G 关键文件完整性指纹	56
附录 H 评审快速核验路径	57
附录 I 术语表	58

摘要

本项目面向云、边、端异构计算资源上的有向无环图（DAG）任务调度问题。输入由带计算量与边通信量的任务图、不同算力和带宽的资源集合组成，输出为每个任务的执行顺序、资源映射与开始/结束时间，优化目标是最小化整体完成时间（makespan）。项目没有把深度强化学习作为不可解释的替代器，而是把它置于可验证的调度物理规则之上：资源时间线统一采用插入式调度，策略只负责在当前就绪任务中确定优先顺序，资源选择由最早完成时间（EFT）规则确定。

技术路线经历了固定奖励导致的策略塌缩、相对 HEFT 奖励、观测归一化、动作空间拆分、残差排序学习和随机多样化推理等阶段。最终方法以零初始化的 Residual Policy 在 HEFT upward rank 上学习有界修正量，使用 Best-of-64 构造初解，随后执行保持前驱约束的单任务重定位，并以 best-only 的破坏—修复大邻域搜索（LNS）进行精修。该精修层只接受严格降低 makespan 的候选，因此对同一初解具有“不退化”保证。

在固定的 20 个验证场景上，决赛优化版最终方案经 5 次系统熵种子的配对重复评测得到 $\text{mean_ratio} = 0.916441 \pm 0.000984$ ，5 轮均在 20/20 个场景反超 HEFT。与同轮原始 Residual Best-of-64 相比，配对差值为 -0.034373 ± 0.003685 ，双侧配对 t 检验 $p = 3.121263 \times 10^{-5}$ 。初赛版 LNS 方案为 0.920890 ± 0.001729 ，仍作为可复核历史基线保留。为排除“仅因计算量增加”的解释，项目另以略高推理耗时的纯 Residual Best-of-128 对齐预算；其结果为 0.942048 ± 0.002063 ，而初赛 LNS 方案仍为 0.920890 ± 0.001729 ，配对检验 $p = 2.072081 \times 10^{-6}$ 。上述数据分别来自 `evaluation/results/final_pipeline_lns_summary.json`、`evaluation/results/autonomous_exploration/direction2_lns/direct_vs_residual_paired_summary.json` 与 `evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json`。

项目同时保留 HEFT、MILP、Hybrid 等基线，提供 Gymnasium 环境、MaskablePPO 训练、固定场景评测、统计审计、Windows/Linux/openEuler 一键运行与测试脚本，并新增 openEuler、openKylin、Anolis OS 的 Docker 矩阵入口。针对决赛阶段最需要回答的“方法是否可用、为何有效、是否泛化、能否复现、能否在国产操作

系统环境运行”五个问题，项目进一步加入宽并行 DAG 专项修复、30~60 任务容量安全策略、单场景调度命令行接口、调度合法性校验、墙钟预算、耗时与进程内存测量。报告如实保留历史失败路线与模型容量边界，不把局部有效结论外推为普遍最优。

从结果口径看，本项目有三组容易混淆但用途不同的数据。第一， 0.916441 ± 0.000984 是最终 Adaptive 方案在原始固定 20 场景、5 次独立重复上的主结果，应作为性能结论首页数字；第二， 0.920890 ± 0.001729 是初赛 Residual+LNS 方案在同一原始验证协议上的历史结果，用于说明决赛优化带来进一步改进；第三， 0.941627 ± 0.026569 是完整系统在 30~60 任务分布外大规模场景上的时间预算结果，用于说明容量安全和工程扩展能力，不能替代原始验证集主结果。三组数值均以越小越优、HEFT 等于 1 为统一解释。

关键词：云边端协同；异构资源调度；DAG；深度强化学习；残差调度；大邻域搜索；可复现性

阅读指南、编写依据与组织说明

本报告面向三类读者：评审专家可先读摘要、1.5 节评分映射、第四章最终算法、第六章实验结果和第十章结论；复现实验人员可直接阅读第七章；希望核查算法细节与数据真实性的读者可结合附录的证据路径、哈希与测试矩阵逐项复核。全文遵循“赛题对齐—实现机制—技术演进—统计证据—复现与边界”的组织方式，所有关键数字都绑定到仓库中的 JSON、日志或测试文件。

1. 赛事官方第 16 题页面明确要求：基于至少一种国内主流开源操作系统开发；提交可训练的 DRL 调度框架；环境与资源参数可配置；提供统一策略接口。评分由功能完整性 40 分、性能优化 35 分、文档质量 25 分组成。本文以 2026 年 7 月 29 日更新的官方赛题页面为首要依据。
2. 第三届赛事初赛作品提交说明要求在仓库根目录提供 README、设计/技术报告、参赛承诺书、演示 PPT 和不超过 100MB 的演示视频，并要求代码、文档、PPT、视频、截图及提交身份匿名。本文使用 docs/references/操作系统开源创新大赛项目说明书_官方模板.docx 的页面规范，但按项目技术逻辑扩充章节。
3. 公开可核验的往届获奖项目 ServerlessOS 与“基于粒子系统的动态图形渲染”说明书普遍采用“问题背景—总体设计—关键实现—实验—使用说明”的主线；本报告借鉴其层次化架构图、关键模块表和复现命令写法，不复制其正文内容。
4. 清华大学《研究生学位论文写作指南》强调论点、数据、图表、单位、参考文献的规范表达；Pineau 等关于机器学习可复现性的研究强调公开配置、随机性、统计口径与运行证据。报告因此把单次结果与重复统计严格分开，并专设“证据索引”和“局限性”章节。

本文对“已验证”“设计保证”和“计划改进”使用不同措辞。“已验证”必须有可定位的原始输出；“设计保证”必须能从候选集合、接受准则或断言直接推出；“计划改进”只列入局限与后续工作，不写成已完成成果。容器验证仅证明三种发行版用户态、Python 与依赖环境中的构建和测试可运行；由于 Docker Desktop 使用 WSL2 共享宿主内核，本文不声称完成三种发行版原生内核或物理机验证。

与旧版相比，第二版不再按实验发生时间简单铺陈，而是先建立数学问题、赛题接口和系统边界，再用诊断链解释技术为何演进，最后将最终结论、反例、复现步骤和原始文件一一绑定。这样既便于评委快速核查，也能避免“结果很好但不知道对应哪段代码”的断裂。

第一章 赛题问题、数学模型与验收映射

1.1 问题定义

设任务图为 DAG $G=(V,E)$ 。节点 $i \in V$ 具有计算量 w_i ，有向边 $(i,j) \in E$ 具有数据量 d_{ij} 。资源集合 R 包含云、边和端节点；资源 r 具有算力 p_r 和带宽 b_r 。调度器需要决定任务执行顺序、资源映射 $x_i \in R$ 、开始时间 s_i 与完成时间 f_i 。

任务 i 在资源 r 上的执行时长为：

$$t_{i,r} = \frac{w_i}{p_r} \quad (1)$$

若前驱 i 与后继 j 分配到不同资源，则通信时延由数据量与有效带宽决定；同资源通信时延为零。任务 j 的开始时间必须满足：

$$s_j \geq \max_{i \in \text{pred}(j)} (f_i + c_{(ij)}(x_i, x_j)) \quad (2)$$

同一资源上的执行区间不得重叠。目标函数为：

$$\min_{x,s} C_{\max}, \quad C_{\max} = \max_i f_i \quad (3)$$

代码对应 `env/dag_generator.py`、`env/resource_config.py`、`env/scheduling_utils.py` 与 `baselines/milp_optimal_scheduler.py`。MILP 基线将资源分配、开始时间与不重叠顺序同时交给求解器，而非预先固定任务顺序。

1.2 默认异构资源

默认资源配置来自 `configs/resource_default.yaml`。

表 1 默认异构资源参数

资源	层级	compute_power	bandwidth
cloud_0	cloud	100	1000
edge_0	edge	30	200
edge_1	edge	25	150
device_0	device	5	20

该配置故意保留显著异构性：最快云节点并不必然产生最短全局 makespan，因为资源竞争、通信代价和 DAG 并行度会改变局部最优选择。

1.3 评价指标

对场景 k ，定义：

$$\rho_k = \frac{C_k^{RL}}{C_k^{HEFT}}, \quad \bar{\rho} = \frac{1}{K} \sum_{k=1}^K \rho_k \quad (4)$$

ratio<1 表示该场景反超 HEFT；ratio=1 表示相同；ratio>1 表示弱于 HEFT。报告同时给出反超场景数、重复评测样本标准差、配对差值与双侧配对 t 检验。评测实现见 `evaluation/evaluate.py`、`evaluation/evaluate_bestofn.py` 与 `evaluation/analyze_residual_ins_direct_repeats.py`。

1.4 赛题要求到代码与证据的闭环

表 2 赛题环节与实现证据对应关系

赛题环节	本项目定义	关键代码	验证证据
问题建模	DAG 节点计算量、边通信量、异构资源算力/带宽	<code>env/dag_generator.py</code> 、 <code>env/resource_config.py</code>	<code>tests/test_env.py</code> 、 <code>tests/test_milp_optimal_scheduler.py</code>
环境状态	任务特征、邻接矩阵、就绪映射、资源状态、全局进度	<code>env/scheduling_env.py</code> 、 <code>env/observation_normalizer.py</code>	<code>tests/test_env.py</code> 、 <code>tests/test_gat_features_extractor.py</code>
动作空间	仅选择当前就绪任务；资源由 EFT 确定	<code>env/scheduling_env_ranked.py</code>	<code>tests/test_scheduling_env_ranked.py</code>
合法性	Action mask + 拓扑就绪条件 + 插入式资源时间线	<code>action_masks()</code> 、 <code>find_earliest_slot()</code>	<code>tests/test_scheduling_consistency.py</code>
奖励	小步惩罚 + 终局相对 HEFT 差距	<code>reward_mode=relative_heft</code>	<code>tests/test_reward_shaping.py</code>
策略	HEFT rank 加零初始化残差 logits	<code>env/scheduling_env_residual.py</code> 、 <code>policies/residual_ranking_policy.py</code>	<code>tests/test_residual_scheduling.py</code>
训练	MaskablePPO，配置驱动	<code>training/train_ppo.py</code> 、 <code>training/configs/ppo_mlp_residual.yaml</code>	训练日志与 checkpoint

赛题环节	本项目定义	关键代码	验证证据
评估	固定场景、HEFT 对照、重复评测、配对 检验	evaluation/	evaluation/results/
理论参照	小规模 MILP 精确/ 限时求解	baselines/milp_optimal_scheduler.py	evaluation/results/milp_optimal_comparison.json
跨平台	Windows、 openEuler、openKylin、 Anolis OS	一键脚本与 Docker 严格 矩阵	evaluation/results/os_matrix.json

1.5 官方评分项逐项验收说明

为避免“报告写了很多，但评委找不到评分证据”，表 2 将官方 100 分评分项拆成可执行验收。这里的“满足”表示仓库已存在相应实现或证据，不表示提前推定评委分数。

表 3 官方评分项逐项验收映射

官方评分项	分值	本项目对应实现	可直接核验的入口
一键训练、验证与结果文件	10	final 复现档自动检查依赖、模型、场景并完成 5 次评测与统计	scripts/reproduce.py、 scripts/run_final_pipeline.ps1、 scripts/run_final_pipeline.sh
模块化与参数抽象	10	DAG、资源、环境、策略、训练、评测彼此解耦；规模、密度、资源通过 YAML/CLI 注入	env/、configs/、training/configs/
外部方法插拔	10	所有调度器实现 BaseScheduler.schedule，HEFT、MILP、Hybrid、RL 共用结果结构	scheduler_interface.py、baselines/、policies/
合法动作处理	10	ready list、action mask、拓扑重建、插入式时间线、输出后校验五层约束	action_masks()、 evaluation/schedule_validation.py、相关测试
验证集指标表现	15	最终 5 次配对重复 0.916441 ± 0.000984，每轮 20/20 反超 HEFT	evaluation/results/final_pipeline_lns_summary.json
泛化能力	10	结构扰动、宽并行修复、30~60 任务大规模容量安全三套实验	evaluation/results/structural_generalization/、 evaluation/results/wide_parallel_adaptive.json、 evaluation/results/final_adaptive_large_scale.json
训练策略有效性	5	记录奖励、归一化、动作拆分、Residual、采样与 LNS 的逐阶段消融和失败路线	第三章及 docs/自主探索日志.md

官方评分项	分值	本项目对应实现	可直接核验的入口
工程效率	5	提供 fast/balanced/quality 三档、软墙钟截止、耗时/P95/RSS 指标	scripts/schedule.py、evaluation/benchmark_scalability.py
赛题对齐说明	10	第一至第五章逐项解释建模、状态、动作、奖励、训练与评估	本报告正文
复现实验说明	10	依赖、版本、命令、配置、种子、输入输出与证据路径齐全	第七章与附录
可读性与结构	5	目录、架构图、流程图、伪代码、三线表、术语表和证据索引	本报告及 docs/figures/

1.6 建模假设与适用范围

本项目解决的是有先后依赖的静态批任务调度。调度开始前，DAG 拓扑、节点计算量、边数据量以及资源算力和带宽已知；单个任务在选定资源上不可抢占；同一资源同一时刻只运行一个任务；两个任务在同一资源上执行时通信代价为零，在不同资源上执行时按数据量和有效带宽估计传输时间。调度结果是一个完整计划，不在任务执行中途重新估计剩余时长。

这些假设使实验能聚焦“依赖、异构算力、通信和资源竞争”四个核心因素，并与 HEFT 的经典模型保持可比。它不覆盖动态任务到达、任务失败重试、资源临时离线、能耗、迁移成本、容器冷启动和多租户公平性。第八章将这些外部有效性限制单独讨论，避免把离线仿真结论直接等同于生产集群效果。

1.7 问题难点与采用混合方法的原因

DAG 调度同时包含离散任务顺序和离散资源分配。若有 n 个任务、 m 个资源，即使忽略依赖，资源映射已有 m^n 种可能；加入所有合法拓扑序、通信延迟和资源时间线后，精确枚举很快不可行。MILP 能在小场景提供理论参照，但在规模上升时求解时间和最优性证明成本显著增加；HEFT 计算快且稳定，却使用固定 upward rank，不能针对每个实例修正局部排序。

因此本项目没有让单一技术承担全部职责：HEFT 提供低成本、可解释的强先验；Residual Policy 学习实例相关偏差；EFT 保证资源放置遵循同一物理规则；LNS 在合

法拓扑序邻域内精修；HEFT anchor 作为最终候选之一提供非回归边界。这一组合兼顾了学习能力、求解质量、可审计性和故障安全。

第二章 系统架构与调度物理规则

2.1 分层架构

统一调度接口贯穿训练、评测与证据链

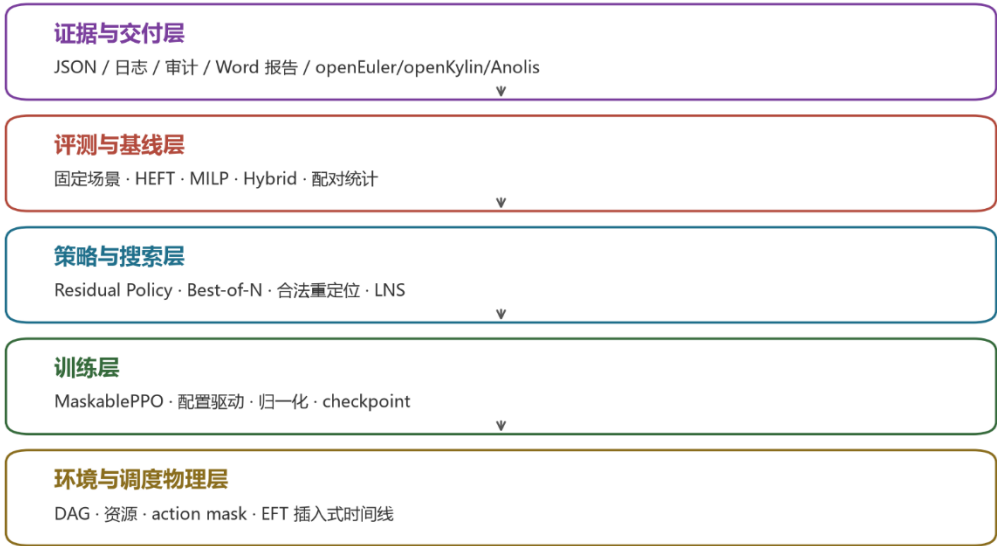


图 1 系统分层架构

项目按职责分为环境层、基线层、策略层、训练层、评测层和证据层。所有调度器继承 scheduler_interface.py 中的统一接口，返回 task_id -> (resource_id, start_time, finish_time)，从而使 HEFT、MILP、Hybrid、RL 和 LNS 在同一 makespan 计算函数下比较。

表 4 项目目录与模块职责

目录	主要职责	代表文件
env/	DAG、资源、观测、奖励、动作掩码与插入调度	env/scheduling_env.py、env/scheduling_env_ranked.py、env/scheduling_utils.py
baselines/	HEFT、MILP、Hybrid	baselines/heft_scheduler.py、baselines/milp_optimal_scheduler.py
policies/	MLP/GAT、Residual、Best-of-N 与搜索精修	policies/residual_ranking_policy.py、policies/residual_local_search_scheduler.py
training/	配置驱动训练、BC、课程学习	training/train_ppo.py、training/behavior_cloning.py
evaluation/	固定场景、泛化、精确解和统计分析	evaluation/evaluate.py、evaluation/analyze_residual_lns_direct_repeats.py
diagnostics/	数据、梯度、mask、归一化与结果审计	diagnostics/bc_dataset_sanity_check.py、diagnostics/technical_report_result_audit.py

目录	主要职责	代表文件
tests/	物理一致性、策略接口与回归测试	tests/test_env.py、 tests/test_scheduling_consistency.py

2.2 插入式资源时间线

初版环境只把新任务追加到 `resource.available_time` 之后，而 HEFT 会搜索资源时间线中的空隙，造成同一分配方案得到不同 `makespan`。项目把这一物理规则抽取为 `env/scheduling_utils.py::find_earliest_slot`，环境和 HEFT 共用 `ScheduledEvent(task_id,start_time,finish_time)` 列表。新任务先计算依赖数据就绪时间，再在已排序事件间寻找第一个足够大的空隙。

一致性测试 `tests/test_scheduling_consistency.py` 将 HEFT 的任务—资源分配按其内部顺序通过环境逐步重放，要求两者 `makespan` 在浮点容差内一致。该测试不是模型效果测试，而是保证比较公平性的基础测试。

2.3 观测空间与归一化

观测为 Gymnasium Dict:

表 5 观测字段及归一化策略

字段	形状	语义	是否归一化
<code>task_features</code>	<code>[max_tasks,5]</code>	有效标志、归一化任务编号、计算量、出度、后继计算量和	连续字段是
<code>task_valid_mask</code>	<code>[max_tasks]</code>	<code>padding</code> 有效位	否
<code>task_adjacency</code>	<code>[max_tasks,max_tasks]</code>	带自环的 0/1 图结构	否
<code>ready_task_node_ids</code>	<code>[max_ready_tasks]</code>	就绪槽位到真实节点编号	否
<code>resource_features</code>	<code>[num_resources,4]</code>	可用时间、层级、算力、带宽	连续字段是
<code>global_features</code>	<code>[2]</code>	完成比例、当前 <code>makespan</code>	是
<code>ready_task_upward_ranks</code>	<code>[max_ready_tasks]</code>	Residual Policy 的 HEFT 基准 logits	按 rank 统计量标准化

`env/observation_normalizer.py` 仅处理连续特征，绝不改变 `mask`、索引和邻接矩阵的离散语义。该边界由 `env/normalization_stats.json` 固化。

2.4 Ranked 动作空间与 EFT 资源选择

Joint 环境每步同时选择“就绪任务槽位 $\times$ 资源”，动作规模随就绪任务数和资源数相乘。Ranked 环境将动作简化为 Discrete(max_ready_tasks)：策略只选“下一个处理哪个就绪任务”。环境随后遍历全部资源，使用相同的依赖就绪时间、执行时长与 find_earliest_slot 计算 EFT，选择完成时间最早的资源。

这一拆分保留了学习模型擅长的全局任务排序，把局部资源放置交给确定性规则。tests/test_scheduling_env_ranked.py 验证：当动作严格复现 HEFT upward-rank 顺序时，最终 makespan 与 HEFT 完全一致。

2.5 相对 HEFT 奖励

每个 episode 在独立资源配置副本上计算 HEFT 参考 makespan，避免污染 RL 环境。奖励为：

$$r_t = -0.01 - \mathbb{I}_{(t=T)} \left(\frac{C_{max}}{C_{HEFT}} - 1 \right) \quad (5)$$

其中，指示函数仅在终局步取 1，其余步骤取 0。该设计把不同任务规模的终局信号统一到相对比值尺度；tests/test_reward_shaping.py 通过重放 HEFT 调度验证终局奖励接近 -0.01。默认 reward_mode=raw 保留旧行为，保证历史实验可复现。

2.6 单个 episode 的状态转移

每个 episode 对应一个完整 DAG。reset() 生成或载入 DAG、深拷贝资源配置、清空资源事件列表，计算入度为零的初始 ready set，并在相对奖励模式下用独立资源副本计算 HEFT 参考值。每次 step(action) 只接受 mask 为真的就绪槽位：环境把槽位映射回真实任务编号，选择 EFT 最小资源，插入任务事件，更新后继剩余入度并把新就绪任务加入集合。全部任务完成时返回终局奖励和最终 makespan。

该流程中“任务编号”“ready 槽位”和“策略动作”不是同一概念。真实任务编号可以稀疏且不按执行顺序排列；策略动作是当前 ready 列表中的位置。项目在环境中显式提供 ready_task_node_ids，并在训练、推理、行为克隆和测试中统一使用这一映射，防止把槽位索引误当作节点编号。

2.7 调度器统一接口与返回契约

所有可比较方法继承 `scheduler_interface.py::BaseScheduler`，核心接口为：

```
[python]
class BaseScheduler(ABC):
    @abstractmethod
    def schedule(self, dag: DAGTask,
                 resource_config: ResourceConfig) -> ScheduleResult:
        ...
```

`ScheduleResult` 的逻辑结构为 `task_id -> (resource_id, start_time, finish_time)`。评测器不读取某个算法的内部状态，而只通过这个结构计算 `makespan`、合法性和相对 HEFT 比值。因此新增策略只要适配同一接口，就能在同一场景、资源副本和指标函数下比较。命令行入口额外把结果展开为按开始时间排序的 JSON 行，便于外部系统消费。

接口设计还规定：调度器不得修改调用方持有的原始资源配置；需要更新可用时间或事件列表时必须使用副本。HEFT reference、学习策略候选和搜索候选分别加载独立资源配置，避免前一个方法的调度状态污染后一个方法。

2.8 调度合法性的五层防线

合法性不是只在最终输出阶段检查，而是贯穿构造、搜索和验收。

1. DAG 载入时检查无环性、节点计算量和边通信量。
2. 环境构造阶段只对 `ready set` 开放 `mask`，非法动作在训练和推理阶段采用同一过滤规则。
3. 单任务重定位只枚举前驱之后、后继之前的位置区间。
4. LNS 修复阶段使用 Kahn 算法，每次只从当前零入度集合中选取任务，因此重建序列天然拓扑合法。
5. `evaluation/schedule_validation.py` 对最终结果再次检查任务是否齐全、资源编号是否存在、时间是否非负、同资源是否重叠、每条依赖是否满足通信就绪时间。

这种“构造时约束 + 搜索时约束 + 输出后验收”的设计比只依赖惩罚项更可靠。非法方案不会因为神经网络打分高而进入候选集；若程序修改破坏约束，测试或运行时校验会直接失败而不是悄悄输出错误 `makespan`。

2.9 公平比较的实现规则

为保证结果可解释，所有方法使用同一 DAG 文件、同一资源配置、同一执行时长与通信时延公式、同一插入式时间线和同一 `makespan` 函数。每个场景先独立运行 HEFT，再运行被测方法；配对重复中同一轮的 Residual 和 LNS 共用采样种子。涉及墙钟时间的实验在同一进程、同一机器上顺序执行，并同时报告算法耗时和 HEFT 参考耗时。

项目不把不同协议下的数字直接排名。例如 0.916441 来自原始 8~25 任务验证集上的五次重复，0.941627 来自 30~60 任务大规模集的 10 秒预算；二者用于回答不同问题。MILP 仅在求解器明确证明 `optimal` 时充当理论最优，超时可行解只记录上下界，不纳入“距最优”均值。

第三章 技术演进：从诊断到残差调度

3.1 演进总览

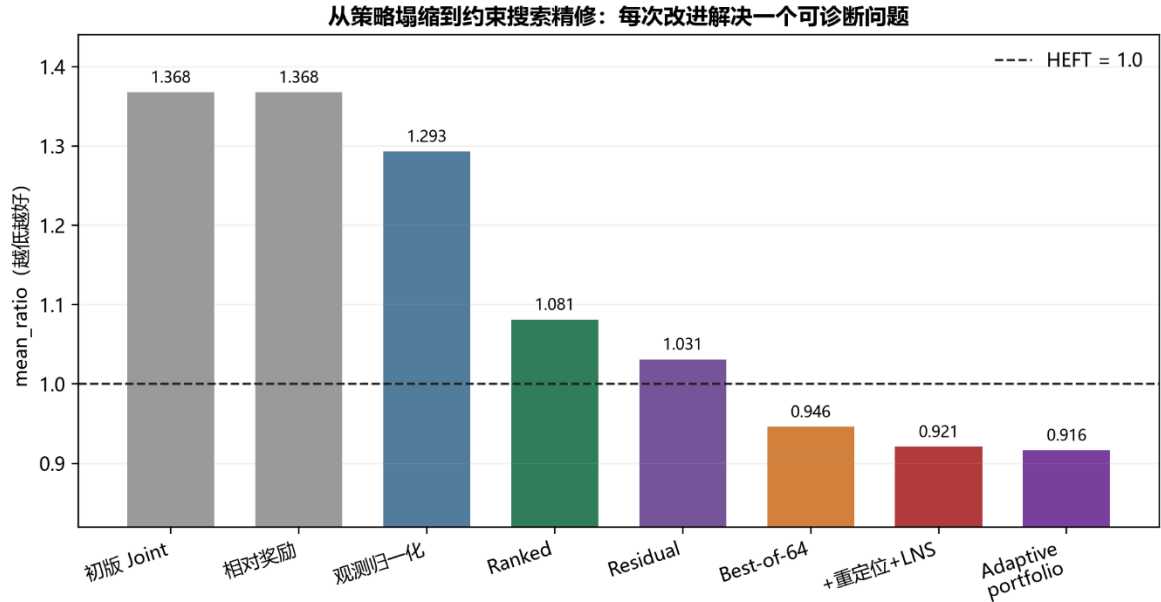


图 2 技术演进与 mean_ratio

表 6 关键技术演进与性能变化

阶段	核心改变	mean_ratio	解决/暴露的问题	数据来源
初版 Joint MLP	固定步惩罚 + 终局绝对 makespan	1.367762	策略塌缩，信号稀疏	evaluation/results/summary.json
Relative HEFT	终局使用相对差距	1.367762	单改奖励仍不足	evaluation/results/summary_reward_shaped.json
观测归一化	连续字段标准化	1.293190	缓解特征量纲冲突	evaluation/results/summary_mlp_normalized.json
Ranked 动作	只学任务顺序，资源用 EFT	1.081277	联合动作难度显著下降	evaluation/results/summary_mlp_ranked.json
Residual	HEFT rank + 有界 delta	1.031197	训练起点锚定 HEFT	evaluation/results/summary_mlp_residual.json
Residual Best-of-64	多次随机构造选优	0.945988	单次策略方差转化为候选多样性	evaluation/results/summary_mlp_residual_bestof64.json
Residual + 搜索精修	合法重定位 + best-only LNS	0.920890 ± 0.001729	修复高质量序列的局部排序错误	evaluation/results/autonomous_exploration/direction2_lns/direct_vs_residual_paired_summary.json

3.2 策略塌缩的发现

初版奖励为每步固定惩罚，终局再减绝对 `makespan`。任务规模变化使终局量纲大幅波动，而每步常数又不能区分动作质量。动作分布诊断显示，10 个诊断 DAG 的 180 个任务全部被分配到 `cloud_0`，证据见 `artifacts/diagnostic_logs/diagnostics_action_distribution_output.log`。这说明模型学到的是“选最快单节点”的短视规则，而非资源并行与通信权衡。

相对 HEFT 奖励修正了尺度，却没有自动解决观测与动作表达问题：对应正式结果仍为 1.367762，见 `evaluation/results/summary_reward_shaped.json`。这一步的价值是排除了“只改奖励即可”的假设。

3.3 归一化诊断

批量 `mask` 检查表明异构合法动作数不会发生跨样本污染；单样本训练也证明梯度、优化器和专家动作索引正常。真正转折来自小数据过拟合：同一批 93 个样本，不归一化的最好准确率为 27.957%，加入字段级标准化后达到 90.323%，证据见 `artifacts/diagnostic_logs/bc_batch_mask_and_normalization_check_output.log`。这说明计算量、带宽、时间与 0/1 特征的尺度差异阻碍了联合优化。

正式加入归一化后 `mean_ratio` 从 1.367762 降至 1.293190，数据分别见 `evaluation/results/summary.json` 与 `evaluation/results/summary_mlp_normalized.json`。归一化解决的是优化条件，而不是动作组合爆炸，因此仍未反超 HEFT。

3.4 动作空间重构

动作复杂度诊断发现，`Joint` 设计把任务排序和资源分配耦合成一个分类问题。`Ranked` 环境保留前者、确定化后者，使动作数从“就绪任务数 $\times$ 资源数”降为“就绪任务数”。结果由 1.293190 进一步降至 1.081277，见 `evaluation/results/summary_mlp_normalized.json` 与 `evaluation/results/summary_mlp_ranked.json`。这是整条技术路线中最关键的结构改进：它改变了学习问题，而不是继续堆叠网络规模。

3.5 残差式排序策略

Residual 环境把每个就绪任务的标准化 HEFT upward rank 暴露为 `ready_task_upward_ranks`。自定义策略 logits 为：

$$z_i = \alpha \tilde{r}_i + \beta \tanh(f_{\theta(o_t, i)}) \quad (6)$$

`policies/residual_ranking_policy.py` 将 `delta` 输出层的权重和偏置全部初始化为零。训练开始时 `delta=0`，策略排序等价于 HEFT；后续 PPO 只学习对 HEFT 的有界修正。测试 `tests/test_residual_scheduling.py` 验证零初始化、logits 加法和合法 mask。

Residual deterministic 的 `mean_ratio` 为 1.031197，优于 Ranked 的 1.081277；对应文件为 `evaluation/results/summary_mlp_residual.json` 与 `evaluation/results/summary_mlp_ranked.json`。它仍不是“保证优于 HEFT”的算法，但提供了更强、可解释的搜索起点。

3.6 Best-of-N 与精修层

`policies/rl_scheduler.py` 在 `num_samples>1` 时对同一 DAG 独立随机构造多个完整调度，保留 `makespan` 最小者，并可加入 deterministic 候选。Best-of-64 的固定种子单次结果为 0.945988，见 `evaluation/results/summary_mlp_residual_bestof64.json`。

最终精修层位于 `policies/residual_local_search_scheduler.py`：

1. 从 Best-of-64 得到最优合法拓扑序。
2. 枚举保持所有前驱—后继关系的单任务重定位，执行 `best-improvement`，最多按配置扫描若干轮。
3. LNS 每次破坏 2 至 4 个任务的位置锚点，用 Kahn 算法重建合法拓扑序，再做一轮局部搜索。
4. 仅当 `candidate_makespan < best_makespan - 1e-12` 时接受；相等或变差候选全部拒绝。

因此“精修后不差于同一初解”是算法设计保证。统计检验验证的是改进幅度在随机重复中是否稳定，而不是证明这个单调性质本身。报告将其定位为“在 Residual 构造器上的 best-only 搜索增强”，不把它包装成与 Residual 平行的独立路线。

3.7 未被采用的方向

负面结果同样决定最终方案边界。

表 7 未采纳方案及消融结论

方向	mean_ratio	结论	数据来源
GA T + Ranked	1.0876 71	未优于 MLP Ranked	evaluation/results/summary_gat_ranked.json
Ranked + BC 热启动	1.1183 87	高 BC 准确率未转化为 PPO 最终优势	evaluation/results/summary_mlp_ranked_bc.json
Ranked + upward rank 输入特征	1.1114 98	“作为输入”不等于“作为强制性基准”	evaluation/results/summary_mlp_ranked_with_rank.json
Ranked + 课程学习	1.1314 71	当前课程区间未改善最终分布	evaluation/results/summary_mlp_ranked_curriculum.json
Gumbel 束 + 相同后处理	0.9209 68 ± 0.001649	相对 LNS 配对 p=0.904282, 未能证明显著改善	evaluation/results/autonomous_exploration/direction3_gumbel_beam/paired_repeats_summary.json

这些结果说明：图编码、模仿学习和课程学习并非无效技术，但在本项目的数据规模、状态表达和训练预算下没有形成可重复优势，因而不进入正式方案。

3.8 与相关方法的关系

HEFT 通过 upward rank 表示从当前任务到出口任务的平均剩余代价，再按优先级和 EFT 完成异构资源调度，是本文最重要的启发式基准。本文保留其平均计算/通信代价定义和 EFT 放置，但把固定 rank 变成可学习的基准项。这样做不是否定 HEFT，而是把它从“最终答案”改为“安全且可解释的先验”。

PPO 使用裁剪代理目标限制策略更新幅度，本项目采用 SB3-Contrib 的 MaskablePPO，使策略分布在采样和训练时都排除非法动作。Residual Scheduling 文献

证明了在组合调度中让学习器预测相对基线修正的可行性；Neural DAG Scheduling 展示了通过优先级采样构造 DAG 调度的思路。本文的差异在于：把 HEFT upward rank 直接注入 logits、把残差头零初始化、把每条采样轨迹转换为完整合法拓扑序，并在统一 EFT 规则下进行多候选和 LNS 精修。

LNS 的核心思想是暂时破坏当前解的一部分，再用约束修复形成跨越小邻域的新候选。本文没有照搬车辆路径问题中的算子，而是针对 DAG 顺序设计“移除 2~4 个位置锚点 + Kahn 拓扑修复”，随后再做一轮单任务重定位。接受策略采用 best-only，不使用退火式接受，因此更适合需要清晰非退化保证的竞赛交付。

3.9 关键设计选择及其取舍

表 8 关键设计选择与工程取舍

设计选择	采用原因	付出的代价	风险控制
Ranked 而非 Joint 动作	显著缩小动作空间，减少任务选择与资源分配耦合	资源选择受 EFT 规则限制	保留统一接口，未来可插入联合策略
Residual 而非从零排序	初始策略等价 HEFT，降低训练冷启动风险	上界受残差尺度和基线质量影响	可调 rank_scale、delta_scale
Best-of-64	把策略随机性转化为候选多样性	推理时间线性增加	fast/balanced/quality 三档和软截止
合法重定位	对高质量序列做确定性精修	邻域仍可能局部最优	LNS 跨越局部邻域
best-only LNS	每次接受都有明确收益，不会破坏现有解	不能越过需要短暂变差的能垒	多初解、多破坏尺度和候选组合
HEFT anchor portfolio	对同一模拟器给出非回归下界	增加少量 HEFT 计算	直接选择统一 makespan 最小候选
容量越界显式跳过模型	防止 40~60 任务张量越界或错误外推	大规模时不使用神经网络	记录跳过原因，改用 HEFT+LNS

3.10 从失败实验得到的工程原则

技术演进得到四条可复用经验。第一，若物理规则不一致，再好的模型结果也不可比较，因此应先统一环境与基线的插入调度。第二，奖励尺度、观测尺度和动作表示是相互关联的，单独修改其中一个不一定产生收益。第三，固定验证集只能回答分布内问题，必须增加结构扰动和大规模分布外场景。第四，机器学习系统应能在模型不适用、文件缺失或时间预算耗尽时给出明确状态和安全结果，而不是依赖“正常情况下不会发生”。这些原则直接推动了后续的容量检查、候选来源字段、输出校验和复现清单。

第四章 最终方案与算法流程

4.1 完整流水线

最终方案：学习构造器 + 约束搜索精修器



只接受严格改善候选：同一初解上 makespan 单调不增

图 3 最终调度流水线

最终方案由“学习构造器”和“约束搜索精修器”串联：

1. 加载 training/checkpoints/ppo_mlp_residual。
2. 对同一 DAG 执行 64 次 stochastic Residual rollout，并加入 deterministic fallback。
3. 每个 rollout 只选择就绪任务，环境用 EFT 插入规则分配资源。
4. 取最小 makespan 的合法序列作为初解。
5. 执行合法单任务重定位至局部最优或达到轮次上限。
6. 决赛优化版同时构造 HEFT 锚点、HEFT+LNS、宽度感知拓扑序和成本优先拓扑序候选，所有候选均重新执行同一 EFT 资源放置。
7. 执行 64 次破坏—修复 LNS；每个候选仍只接受严格改善。
8. 在所有合法候选中选择 makespan 最小者，输出最终任务顺序、资源、开始与结束时间。

4.2 正确性与不变量

表 9 最终搜索阶段的不变量与保障机制

不变量	保障机制	测试/检查
只调度就绪任务	环境 ready list 与 action mask	tests/test_scheduling_env_ranked.py

不变量	保障机制	测试/检查
输出保持拓扑合法	重定位前检查边两端位置；修复使用 Kahn 算法	tests/test_residual_local_search_scheduler.py
同资源任务不重叠	find_earliest_slot 插入式时间线	tests/test_scheduling_consistency.py
环境与 HEFT 物理规则一致	共用 ScheduledEvent 和插入函数	tests/test_scheduling_consistency.py
Residual 初始策略锚定 HEFT	delta 输出层零初始化	tests/test_residual_scheduling.py
搜索不劣化输入解	best-only 接受 + 运行时断言	tests/test_residual_local_search_scheduler.py

4.3 算法定位

该方法不是用神经网络直接预测连续开始时间，也不是用 MILP 求解全部规模。它将三种能力组合：HEFT 提供结构化先验，Residual Policy 学习实例相关排序偏差，LNS 在合法解空间内做计算预算可控的单调精修。资源放置始终由 EFT 确定，因此模型和搜索都不会绕过操作系统调度约束。

4.4 Residual 构造阶段伪代码

```
[text]
输入：DAG G, 资源 R, Residual 模型  $\pi$ , 采样数 N
输出：当前最优拓扑序 order_best 与调度 schedule_best

初始化 best_makespan =  $+\infty$ 
重复 N 次：
    reset(G, copy(R))
    当仍有未调度任务时：
        ready = 当前零入度且未执行的任务
        base_logits = 标准化 HEFT upward rank(ready)
        delta = tanh( $\pi(\text{observation}, \text{ready})$ )
        logits = rank_scale * base_logits + delta_scale * delta
        task = 从 mask 后的 logits 随机采样
        resource = argmin_r EFT(task, r)
        将任务插入该资源最早可行空隙并更新 ready
    若 makespan 更小，则保存完整序列与调度
额外运行一次 deterministic rollout 作为稳定候选
返回 makespan 最小的合法候选
```

残差头最后一层初始权重与偏置为零，因此第一次前向计算中的 $\delta=0$ 。随着 PPO 训练，网络学习的是在不同全局进度、资源占用和任务结构下，对 HEFT rank 的有限修正。 $\tanh$ 防止单个异常输出无限放大并完全抹去先验。

4.5 单任务重定位与 LNS 伪代码

```
[text]
输入：合法拓扑序 order，最大重定位轮数 P，LNS 轮数 L
current = order
对 pass = 1..P:
    枚举每个任务的所有合法插入位置
    用统一 EFT 重放并计算每个候选 makespan
    选择本轮严格改善最大的候选；若不存在则停止

best = current
对 iteration = 1..L:
    随机选择 2~4 个任务并移除其位置锚点
    用 Kahn 算法在前驱约束下修复成完整拓扑序
    对修复序列再执行一轮合法重定位
    仅当 candidate_makespan < best_makespan - 1e-12 时接受
返回 best
```

重定位不是任意交换。对任务 v ，合法插入区间左边界由所有前驱的当前位置决定，右边界由所有后继决定；只在这个区间内尝试新位置。LNS 修复时每次从零入度集合中选任务，因此不会生成含逆序依赖的候选。每个序列最终都通过 `schedule_task_order` 重新进行资源放置，不沿用可能过期的开始/结束时间。

4.6 自适应候选组合与宽并行修复

`ResidualLargeNeighborhoodScheduler` 首先计算任务数、边数、源点数、汇点数、最长路径、最大层宽、宽度比例与边密度。任务数不超过模型容量时，`Residual+LNS` 是候选之一；无论图形如何，都会加入精确 HEFT anchor，并在预算允许时加入 HEFT+LNS。若宽度比例不小于默认阈值 0.30，则进一步加入宽度感知和成本优先拓扑序。

宽度感知顺序不是简单按层号排序。算法每步仍只从当前就绪集合选任务，但在 HEFT rank 之外提高高计算量、高出度任务的优先度，减少宽前沿中大量同级任务仅靠固定 rank 发生的局部拥塞。成本优先候选使用 rank_weight=0.15、cost_weight=0.75、fanout_weight=0.10，与默认宽度候选形成不同搜索起点。所有候选经过同一重定位与 LNS，并按同一 makespan 比较。

4.7 非回归、容量安全与时间预算

对 Adaptive 模式，设候选集合包含 S_{HEFT} ，最终输出为 $\operatorname{argmin}_{\{S \in \text{Candidates}\}} C_{\max}(S)$ ，则在浮点容差和同一模拟器下有：

$$C_{\max}^{(Adaptive)} \leq C_{\max}^{HEFT} \quad (7)$$

该不等式来自选择规则，不依赖统计显著性，也不要求 Residual 模型每次都优于 HEFT。实现还在大规模基准中设置运行时断言：任何场景若 $\text{ratio} > 1 + 10^{-9}$ 则立即失败。

最终 checkpoint 的 max_tasks_padding=30。当 DAG 超过 30 个任务时，系统不尝试裁剪、补丁式扩展或将越界张量送入模型，而是记录 task_count=... exceeds residual_model_capacity=30，跳过 Residual 候选并继续 HEFT+LNS 安全路径。这样把“完整系统容量”与“神经模型容量”分离，确保能处理 40~60 任务且不会作不受支持的神经网络外推声明。

软墙钟预算在 rollout、重定位候选和 LNS 迭代之间检查。预算到达后不再启动新的昂贵候选，但已开始的原子计算和 HEFT anchor 允许完成，因此它是软截止而非硬杀进程。命令行默认提供三档：fast=8/1/8、balanced=32/2/32、quality=64/3/64，三个数字依次表示 Residual 采样数、重定位轮数和 LNS 迭代数。

4.8 时间复杂度与资源开销分析

设任务数为 n 、资源数为 m 、Residual 采样数为 N 、重定位轮数为 P 、LNS 轮数为 L 。一次给定拓扑序的 EFT 重放需要处理每个任务、资源和时间线空隙，朴素上界可写为 $O(n \cdot m \cdot n)$ ；单轮全邻域重定位最多评估 $O(n^2)$ 个序列，因此最坏时间由重复

重放主导。LNS 每轮包含一次修复、一次重放和有限重定位，整体近似随 N 、 P 、 L 线性增长，但随 n 呈高阶增长。

项目不只给渐进符号，还在第 6.5 节报告实测墙钟、P95、进程 CPU 与峰值 RSS。固定工作量的 30~60 任务测试显示平均 6.633 秒、P95 13.284 秒、峰值约 359.1 MiB。该量级适合秒级离线批调度或计划生成，不适合作为毫秒级硬实时控制器。性能档位和时间预算允许工程部署在质量与延迟之间做明确选择。

4.9 异常与降级路径

表 10 异常与安全降级行为

异常或边界条件	系统行为	是否继续输出
DAG 有环或字段非法	输入校验失败并给出错误	否
checkpoint 不存在或不兼容	模型加载阶段失败，不伪装为 RL 成功	否；可显式选择 <code>--method heft</code>
任务数超过模型容量	跳过 Residual，记录原因，使用 HEFT+LNS	是
时间预算耗尽	停止启动新搜索，保留已有最优和 HEFT anchor	是
搜索候选变差	best-only 拒绝；运行时断言防止 LNS 反向劣化	是
输出任务缺失、重叠或违反依赖	schedule_validation 判失败	否
Docker 不可用	OS 矩阵标记失败并保留诊断，不修改算法结果	本地评测可独立运行

第五章 实验协议与统计方法

5.1 数据、训练与验证隔离

最终训练配置为 `training/configs/ppo_mlp_residual.yaml`：训练 DAG 任务规模为 8 至 25，边密度范围为 0.2 至 0.5，padding 上限为 30，训练步数配置为 200000，随机种子为 123。固定验证集包含 20 个场景，任务规模为 10、15、20、25，每组 5 个，验证种子从 1000000 开始。配置与场景路径分别见该 YAML 和 `evaluation/scenarios/`。

报告区分三种随机性：训练随机种子、固定复现实验采样种子、系统熵重复评测种子。最终统计结论使用系统熵种子，不从多次运行中挑选最好结果。

5.2 基线

1. HEFT：使用 upward rank 排序和 EFT 插入式资源选择，见 `baselines/heft_scheduler.py`。
2. MILP：同时优化分配与时间，CBC 限时求解；只有状态为 `optimal` 的场景才用于“距理论最优”统计，见 `baselines/milp_optimal_scheduler.py`。
3. Hybrid：小规模优先 MILP，失败或超阈值回退 Residual+Best-of-N，见 `baselines/hybrid_scheduler.py`。
4. 纯采样计算量对照：Residual Best-of-128，其平均耗时略高于最终 LNS，用于排除计算量混淆。

5.3 重复与配对设计

最终方案与 Residual Best-of-64 使用相同的 5 个系统熵种子，每轮的 20 个场景共享该轮种子。令 $d_i = \text{mean_ratio_new}_i - \text{mean_ratio_baseline}_i$ ，报告样本均值、样本标准差和双侧配对 t 检验。显著性阈值为 0.05；只有均值更低且 $p < 0.05$ 才记为显著改善。

计算量对齐实验先只根据耗时校准 N：N=84 耗时偏低后，按比例选择 N=128；达到目标耗时范围后锁定，不根据 `mean_ratio` 再调 N。校准与正式数据保存在 `evaluation/results/autonomous_exploration/compute_matched_sampling/`。

5.4 场景生成、划分与分布外评测

常规训练 DAG 每轮按配置在 8~25 个任务、0.2~0.5 边密度范围内生成；训练生成器使用种子 123 驱动的独立随机序列。固定验证集不在训练中回放，包含 10、15、20、25 任务各 5 个，种子从 1000000 开始。所有方法读取同一组持久化 JSON，避免每次评测重新生成不同图。

结构化泛化集使用全新种子并构造四组：宽并行、深链条、同构资源和原始分布对照。宽并行图控制较短最长路径和更大并发前沿；深链条强调长依赖链；同构组与原始对照共享 DAG 但替换资源配置，用于隔离资源异构性的影响。历史结果先暴露宽并行弱点，决赛优化再使用独立的 `scenarios_structural_v2/wide_parallel` 进行专项修复评测。

大规模集包含 30、40、50、60 任务各 5 个，使用固定种子 20260905 起的场景序列。30 任务位于模型 padding 上限，40~60 明确超出神经网络容量。该设计用于同时验证边界点、越界降级、耗时增长和最终调度合法性，不与原始验证集的模型泛化指标混为一谈。

5.5 指标定义与统计解释

主指标 `mean_ratio` 先对每个场景计算 `method_makespan/HEFT_makespan`，再对场景取算术平均。相对 HEFT 的平均改善百分比为 $(1-\text{mean_ratio}) \times 100\%$ 。由于分母随场景变化，本文不直接用 `makespan` 的绝对差跨场景平均；绝对值仍保存在逐场景 JSON 中供核查。

“±”统一表示独立重复或场景样本的样本标准差，不表示标准误。原始 20 场景主结论的统计单位是 5 次完整评测，每次包含相同 20 个场景；大规模结果的统计单位是 20 个不同场景。二者样本单位不同，因此标准差不能直接比较。

配对 t 检验用于同一随机种子、同一场景集合下的两个方法。零假设为配对差值均值为零，双侧 $p < 0.05$ 表示差值显著偏离零；是否“更好”还要求差值方向为负。报告同时给出效应方向、反超场景数和原始数据，不把 p 值当作实际收益大小。样本量较小时，结论限于当前协议，不声称代表所有 DAG 分布。

5.6 实验控制与可复现措施

表 11 实验控制与可复现措施

控制项	实施方式	目的
场景控制	验证 DAG 持久化为 JSON	所有策略读取完全相同输入
资源控制	每次调度加载独立 YAML 副本	防止资源可用时间跨方法污染
随机控制	Python、NumPy、PyTorch 同步设种子	降低采样不可复现性
代码控制	结果记录配置路径、模型路径和 commit	建立结果到实现的溯源
统计控制	先固定协议再读取结果；保留所有重复	避免挑选最好一次
合法性控制	每个大规模输出执行完整校验	防止以非法调度换取低 makespan
平台控制	三个容器执行同一 compile/test/generate 命令	验证用户态依赖兼容性
隐私控制	日志中的主目录替换为 <USER_HOME>	避免证据文件泄露身份

PyTorch 官方文档指出，跨版本、跨平台以及 CPU/GPU 之间不保证逐位相同。本文因此把“复现”定义为：输入、配置、种子、算法路径和统计脚本固定，输出满足同一合法性与指标口径；不承诺不同处理器上的随机采样浮点序列逐位一致。正式主结果保留已运行的原始 JSON，不在生成 Word 时重新计算或改写。

5.7 测试策略

测试分为四层。单元测试验证 DAG、观测、mask、Residual logits、时间线和统计辅助函数；契约测试验证统一调度接口与 JSON 输出；性质测试验证拓扑合法和 best-only 非退化；集成测试执行训练模型加载、调度器运行、MILP/HEFT 对比和三发行版容器 smoke。

本地最新完整测试为 42 passed, 17 warnings。OS 矩阵形成时每个容器运行的是当时提交的 38 项测试，因此原始 evaluation/results/os_matrix.json 如实保留 38 passed, 17 warnings，不会用后续本地新增 4 项测试覆盖历史记录。17 条警告来自 PuLP 未来版本接口弃用提示，不是当前求解失败；报告在局限性中保留升级计划。

第六章 结果与证据

6.1 最终方案的五次重复结果

表 12 最终方案五次重复评测结果

重复	Residual Best-of-64	最终 LNS	差值 (LNS- Residual)	Residual 反超数	LNS 反 超数	总耗时/秒
1	0.949533	0.917625	-0.031908	18	20	123.950186
2	0.948910	0.915701	-0.033210	19	20	121.122092
3	0.956776	0.915895	-0.040881	17	20	122.065939
4	0.948961	0.915591	-0.033370	19	20	122.376658
5	0.949891	0.917393	-0.032498	18	20	129.194600
均值 ±	0.950814	0.916441	-0.034373	18.2/20	20.0/20	123.741895
样本标准差	± 0.003358	± 0.000984	± 0.003685			

双侧配对 t 检验 $t=-20.859262$ 、 $df=4$ 、 $p=3.121263 \times 10^{-5}$ 。完整逐轮数据与统计量来自 `evaluation/results/final_pipeline_lns_summary.json` 与 `evaluation/results/final_pipeline_lns_repeats/`。

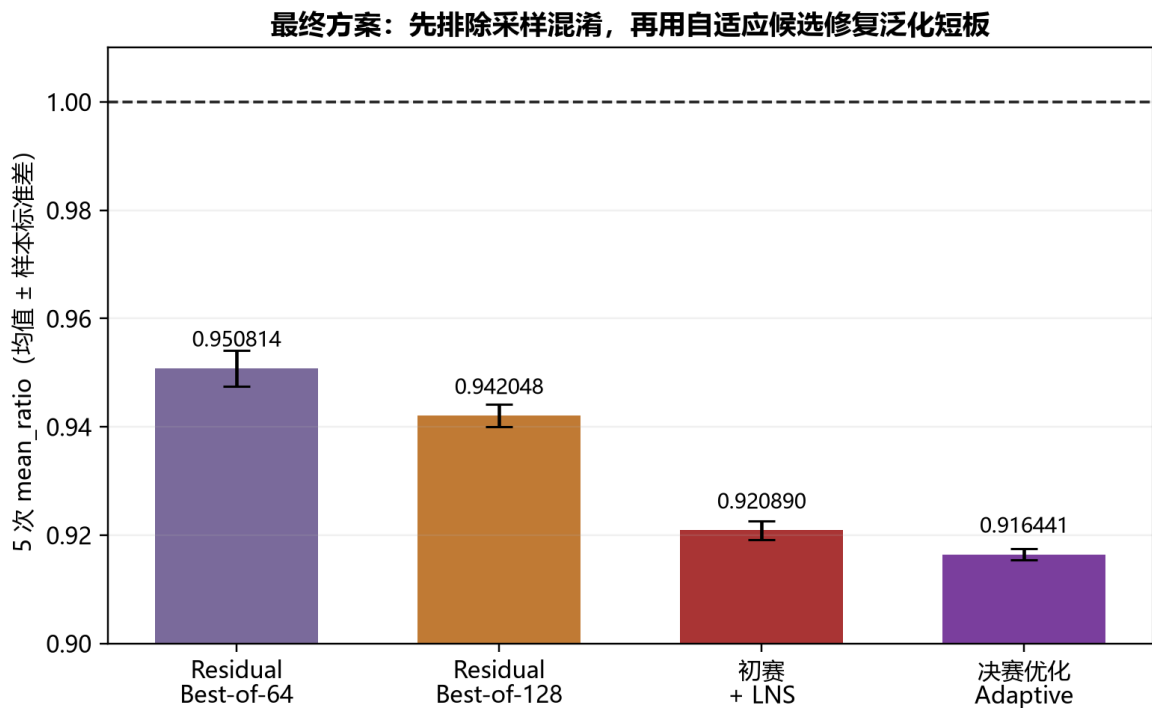


图 4 最终方案重复统计

6.2 计算量对齐验证

表 13 计算量对齐对比结果

指标	纯 Residual Best-of-128	初赛 Residual Best-of-64 + 重定位 + LNS
mean_ratio (5 次均值 $\pm$ 样本标准差)	0.942048 $\pm$ 0.002063	0.920890 $\pm$ 0.001729
平均反超 HEFT 场景数	18.6/20	20.0/20
20 场景平均耗时/秒	56.801547	54.401734

LNS 减纯采样的配对差值为 -0.021158 ± 0.001148 ，双侧配对 t 检验 $t=-41.210742$ 、 $df=4$ 、 $p=2.072081 \times 10^{-6}$ 。纯采样耗时为 LNS 的 1.044113 倍，却仍明显更差。数据来自 `evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json`。因此，在当前验证集与硬件上，改进不能仅由增加 wall-clock 预算解释；LNS 的邻域结构带来了额外价值。决赛优化版进一步加入启发式候选组合，主要目标是修复结构泛化短板，因此其耗时高于该计算量对照，不把两者作为同预算比较。

6.3 与 MILP 理论最优的距离

小规模 10/15 任务场景共 10 个，CBC 在限定时间内对其中 7 个证明全局最优，3 个仅得到未证明最优的可行解。仅统计 7 个已证明最优场景：

表 14 MILP 最优解距离对比

方法	方法 makespan / MILP optimum	高于最优解
HEFT	1.106807	10.6807%
Residual+Best-of-16	1.044188	4.4188%

数据来自 `evaluation/results/milp_optimal_comparison.json`。该实验早于最终 LNS，因此只证明学习构造方法在可精确求解规模上比 HEFT 更接近理论下界，不把 Best-of-16 数字冒充最终 LNS 的最优差距。3 个未证明最优场景保留 incumbent、best bound 与 gap，不纳入均值。

6.4 结构化泛化

结构化泛化使用全新种子、保持任务数在训练范围内，并改变 DAG 密度或资源异构性。该实验评测的是 Residual Best-of-64，而非后续 LNS；它用于描述学习模型本身的结构迁移能力。

表 15 结构化泛化评测结果

场景组	mean_ratio	场景内标准差	反超 HEFT	相对原始对照差值
宽并行	1.011915	0.018469	2/5	+0.047829
深链条	0.943597	0.043357	4/5	-0.020488
同构资源	0.879572	0.071489	5/5	-0.084514
原始分布对照	0.964085	0.040774	4/5	0.000000

数据来自 `evaluation/results/structural_generalization/summary_structural_generalization.json`。宽并行组退化至略弱于 HEFT，说明当前训练分布对大量并发就绪任务覆盖不足；同构资源组明显更好，说明资源差异缩小后，任务排序优势更容易转化为 makespan 改善。

针对该短板，决赛优化版新增宽并行 DAG 生成器、结构混合训练配置和运行时自适应候选集。正式宽并行 64/64 评测在 `evaluation/scenarios_structural_v2/wide_parallel` 的 5 个场景上达到 `mean_ratio=0.929848`，5/5 不劣于 HEFT；其中 4/5 严格反超，剩余 1 个打平场景由 MILP 证明当前 makespan 已是全局最优。证据见 `evaluation/results/wide_parallel_adaptive.json`。该修复不删除历史负面结果，而是把“发现短板—增加候选机制—重新评测—证明打平原因”作为闭环保留。

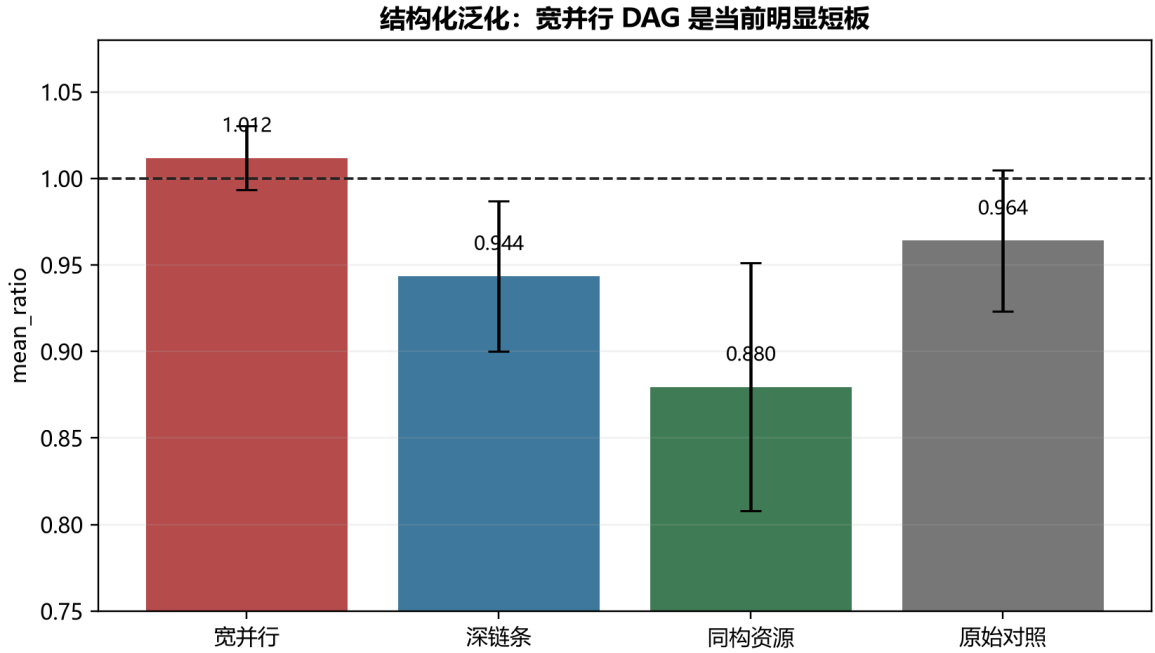


图 5 结构化泛化

6.5 大规模泛化与容量安全

为避免把“换模型”和“换规模”混为一谈，项目曾另行训练支持 60 个任务、padding 70 的归一化 MLP。该模型在原规模评测上的 mean_ratio 为 1.336977，在 30 至 60 任务的大规模集上为 1.454469，分别见 evaluation/results/summary_mlp_normalized_large_small_eval.json 与 evaluation/results/summary_mlp_normalized_large.json。该历史负面结果没有支持“任务越大越容易反超 HEFT”的假设，说明仅扩大 padding 和训练范围不能解决规模泛化。

决赛优化版进一步把“神经网络容量”与“调度系统容量”分开处理。最终 Residual checkpoint 的输入容量为 30；当任务数超过 30 时，调度器不会把越界输入送入模型，而是记录跳过原因并使用 HEFT 锚点与 best-only LNS。最终候选仍按同一 EFT makespan 取最优，因此完整系统对 HEFT 保持非回归。固定 30/40/50/60 任务场景各 5 个，结果如下：

表 16 大规模泛化与运行开销

协议	mean_ratio ± 样本标准差	反超 HEFT	双侧 p 值	平均耗时	P95 耗时	峰值 RSS
64/3/64, 每场景 10 秒 软预算	0.941627 ± 0.026569	20/20	6.985758×10 ⁻⁹	10.002 秒	10.003 秒	358.1 MiB

协议	mean_ratio ± 样本标准差	反超 HEFT	双侧 p 值	平均耗时	P95 耗时	峰值 RSS
8/1/8, 固定工作量无截止	0.949723 ± 0.023441	20/20	1.025617×10 ⁻⁸	6.633 秒	13.284 秒	359.1 MiB

10 秒质量档在 30、40、50、60 四组的 mean_ratio 分别为 0.934907、0.926731、0.948592、0.956278，各组均 5/5 严格反超 HEFT。固定工作量协议的平均耗时从 30 任务 3.641 秒增长到 60 任务 13.188 秒。原始逐场景耗时、CPU 时间、RSS、候选来源及合法性检查见 `evaluation/results/final_adaptive_large_scale.json` 和 `evaluation/results/final_adaptive_large_scale_fixed.json`。

该实验支持“完整 Adaptive 系统可安全扩展到 60 任务”，但不把启发式安全路径包装成神经网络规模泛化：30 任务仍可使用 Residual 候选，40 至 60 任务明确记录为跳过模型并由 heft_ins 产生最终结果。

6.6 openEuler 与测试验证

项目在 openEuler 24.03 LTS-SP4 容器中挂载同一代码与 checkpoint，完成依赖安装、pytest 和评测。保留证据包括 `evaluation/results/openeuler_validation/environment_info.log`、`evaluation/results/openeuler_validation/pip_freeze.txt`、`evaluation/results/openeuler_validation/pytest_output.log` 与 `evaluation/results/openeuler_validation/evaluate_bestof64_output.log`。其中历史结构化测试记录为 19 passed, 17 warnings in 5.33s，见 `evaluation/results/openEuler_pytest_structural_final.log`。

决赛优化版新增 `docker/Dockerfile.openeuler`、`docker/Dockerfile.openkylin`、`docker/Dockerfile.anolis` 和 `scripts/run_os_matrix.py`，并使用对应社区维护的基础镜像仓库，在 openEuler 24.03 LTS-SP4、openKylin 2.0 SP1 与 Anolis OS 23 容器中执行统一严格矩阵。2026-08-31 的 `python scripts/run_os_matrix.py --pull --strict` 真实运行结果如下：

表 17 国产操作系统容器验证

系统	Python	PyTorch	C UDA	构建退出码	运行退出码
openEuler 24.03 LTS-SP4	3.11.6	2.12.1+cpu	False	0	0

系统	Python	PyTorch	CUDA	构建退出码	运行退出码
openKylin 2.0 SP1	3.12.2	2.12.1+cpu	False	0	0
Anolis OS 232	3.10.12	2.12.1+cpu	False	0	0

三套容器均完成系统身份记录、CPU-only PyTorch 校验、全项目字节码编译、完整 pytest 与 20 个结构泛化场景生成；每套容器的测试结果均为 38 passed, 17 warnings, 机器可读原始输出见 evaluation/results/os_matrix.json。同日 Windows 本地一键 smoke 运行结果同样为 38 passed, 17 warnings。警告来自 PuLP 已标注的未来接口弃用，不影响当前测试结论。该证据验证的是代码与推理入口的跨平台可运行性，不声称不同 CPU 上的随机采样结果逐位一致，也不声称已在 OpenHarmony 原生设备系统中运行。

6.7 主结果、历史结果与大规模结果的统一解释

表 18 多组核心结果的口径区分

数值	实验对象	样本单位	回答的问题	是否作为首页主结果
0.920890 ± 0.001729	初赛 Residual Best-of-64 + 重定位 + LNS	5 次×20 固定场景	初赛方案是否稳定优于 HEFT	否，作为历史基线
0.916441 ± 0.000984	决赛 Adaptive portfolio + 重定位 + LNS	5 次×20 固定场景	最终方案在验证协议上的性能	是
0.929848	宽并行专项修复	5 个独立宽并行场景	历史弱点是否形成闭环修复	专项结果
0.941627 ± 0.026569	30~60 任务、10 秒软预算	20 个大规模场景	完整系统能否安全扩展并限时输出	工程泛化结果
0.949723 ± 0.023441	30~60 任务、8/1/8 固定工作量	20 个大规模场景	不用截止时的固定计算量开销	工程对照

数值越低表示相对 HEFT 越好。因此 0.916441 相比 0.920890 是进一步改善而不是退步；两者在相同原始验证协议下可比较。0.941627 虽然也小于 1，但任务规模、统计单位和预算不同，只能说明完整系统在新场景中平均比 HEFT 缩短约 5.84%，不能与 0.916441 直接排名模型优劣。

6.8 改进幅度与稳定性

最终主结果 0.916441 对应相对 HEFT 平均缩短约 8.3559%。五次完整评测的范围为 0.915591~0.917625，样本标准差仅 0.000984；这说明在固定场景与五个采样种子

下，最终组合对随机 rollout 的敏感度较小。相比同轮 Residual Best-of-64，平均差值 -0.034373，即在 ratio 尺度上再改善 3.4373 个百分点。

初赛 LNS 0.920890 对 HEFT 平均改善约 7.9110%；决赛 Adaptive 进一步降低约 0.4449 个 ratio 百分点。提升看似小于从原始 Residual 到 LNS 的跃迁，但它同时修复宽并行非回归、加入 HEFT 安全锚点、容量检查和时间预算，因此价值不仅体现在均值。

大规模 10 秒质量档 20/20 严格反超 HEFT，95% 置信区间为 [0.929192, 0.954062]，双侧检验 $p=6.985758 \times 10^{-9}$ 。固定工作量档同样 20/20 严格反超，均值 0.949723。对于 40~60 任务，最终来源是 heft_lns，这证明搜索增强路径有效，而非证明 30 容量的神经模型完成了规模外泛化。

6.9 负面证据与结论边界

本项目保留三类负面证据。其一，初版 Joint MLP 为 1.367762，表明直接学习“任务×资源”联合动作失败；其二，历史宽并行 Residual Best-of-64 为 1.011915，表明原模型对宽前沿泛化不足；其三，专门扩大 padding 到 70 的历史 MLP 在大规模集为 1.454469，表明单纯扩大输入容量不等于规模泛化。

负面结果未从仓库删除，也未在最终组合选优中冒充有效候选。它们用于说明为什么最终系统采用动作拆分、Residual 先验、宽度感知候选和显式容量降级。最终可以稳健声称的是：当前固定资源模型和验证协议下，完整 Adaptive 系统在常规、宽并行专项和 30~60 任务大规模集上均未出现弱于 HEFT 的正式场景；不能声称任意 DAG、任意资源配置、动态任务或原生国产操作系统内核上都必然达到同等比例。

第七章 复现、审计与工程交付

7.1 依赖与关键配置

Python 依赖由 requirements.txt 声明并锁定直接依赖版本：NetworkX、Gymnasium、PyYAML、NumPy、Matplotlib、pytest、SB3-Contrib、Stable-Baselines3、TensorBoard、tqdm、Rich、PuLP 和 SciPy。该版本集同时兼容 Python 3.10、3.11、3.12；CPU-only PyTorch 另固定为 2.12.1+cpu。最终训练参数、环境范围、checkpoint 与结果路径以 training/configs/ppo_mlp_residual.yaml 为准。

表 19 复现配置与用途

配置/文件	用途
configs/resource_default.yaml	默认云边端资源
env/normalization_stats.json	连续观测字段统计量
training/configs/ppo_mlp_residual.yaml	Residual 训练与常规评测
training/checkpoints/ppo_mlp_residual.zip	最终训练模型
evaluation/scenarios/	固定常规验证集
evaluation/results/autonomous_exploration/	最终搜索与统计证据

7.2 一键运行

Windows PowerShell:

```
[powershell]
powershell -ExecutionPolicy Bypass -File scripts/run_final_pipeline.ps1
```

Linux/openEuler:

```
[bash]
bash scripts/run_final_pipeline.sh
```

现有一键脚本检查依赖与 checkpoint，必要时训练 Residual 模型，生成固定场景，并使用正式报告中的 5 个规范种子运行完整的 Adaptive portfolio、Residual Best-of-64、合法拓扑序重定位和 best-only LNS 流程。脚本随后自动执行配对统计，输出 mean_ratio=0.916441 ± 0.000984 及 5/5 轮均为 20/20 场景反超 HEFT。规范种子最初来

自系统熵采样，复现脚本将其固定，避免不同运行因随机采样产生口径漂移。统计汇总保存在 `evaluation/results/final_pipeline_lns_summary.json`，逐轮 JSON 保存在 `evaluation/results/final_pipeline_lns_repeats/`。

单场景系统接口：

```
[bash]
python scripts/schedule.py \
  --input evaluation/scenarios/scenario_10_0.json \
  --output evaluation/results/schedule_output.json \
  --preset balanced \
  --time-budget-seconds 10
```

输出 JSON 包含任务—资源映射、开始/结束时间、makespan、HEFT 对照、候选来源、耗时、进程内存和调度合法性。批量大规模基准由 `evaluation/benchmark_scalability.py` 执行。

7.3 单次 LNS 调试命令

```
[bash]
python evaluation/evaluate_residual_lns.py \
  --config training/configs/ppo_mlp_residual.yaml \
  --model-path training/checkpoints/ppo_mlp_residual \
  --num-samples 64 \
  --lns-iterations 64
```

如需复现 5 次统计，应使用系统熵生成独立种子，逐轮保存 JSON，再运行：

```
[bash]
python evaluation/analyze_residual_lns_direct_repeats.py \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_1.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_2.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_3.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_4.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_5.json
```

7.4 测试与报告审计

```
[bash]
python -m pytest tests/ -v
python diagnostics/technical_report_result_audit.py --report docs/技术报告.md
```

审计器检查报告引用路径是否存在、数字是否与 JSON 字段在显示精度内一致、参考文献 URL 是否在核验清单中，并扫描可识别的用户主目录路径。Word 文档由 `scripts/generate_report_docx.py` 通过 win32com 直接操控 Microsoft Word 生成，随后更新目录、页码和域并导出 PDF 做页面核验。

7.5 统一复现入口与三个档位

跨平台入口 `scripts/reproduce.py` 提供 `smoke`、`wide`、`final` 三个 profile。`smoke` 运行字节码编译、完整测试和结构场景生成，适合首次安装后快速验收；`wide` 在 `smoke` 基础上执行宽并行 Adaptive 对比；`final` 使用已固定的五个规范种子运行完整最终流水线并生成汇总。`--write-manifest` 会记录解释器、平台、命令、退出码和耗时；`--with-os-matrix` 可继续构建三套国产操作系统容器。

```
[bash]
# 约数分钟内检查代码、测试和场景生成
python scripts/reproduce.py --profile smoke --write-manifest

# 验证宽并行 DAG 修复，可用较小参数做快速演示
python scripts/reproduce.py --profile wide --num-samples 16 --lns-iterations 16

# 正式五次重复，耗时较长，生成最终统计文件
python scripts/reproduce.py --profile final --write-manifest
```

复现入口不会把工作目录的绝对用户路径写入公开 manifest，Windows 与 Linux 主目录会被替换为 `<USER_HOME>`。每个子命令的实际参数和退出码都写入机器可读记录，任一必须步骤失败时总流程返回非零退出码。

7.6 单场景真实调度接口

`scripts/schedule.py` 是面向实际使用而非论文统计的入口。输入是 NetworkX node-link JSON DAG，节点包含 `computation`，边包含 `data_size`；资源从 YAML 读取。输出不仅有 `ratio` 汇总，还含每个任务的资源、开始时间与完成时间，因此可用于演示、下游执行器或可视化。

```
[powershell]
.\.venv\Scripts\python.exe scripts\schedule.py `
  --input evaluation\scenarios\scenario_10_0.json `
  --output evaluation\results\schedule_output.json `
  --method adaptive `
  --preset balanced `
  --time-budget-seconds 10 `
  --seed 20260905
```

命令完成时终端打印 `SCHEDULE_COMPLETE`、候选来源、任务数、`makespan`、`ratio`、耗时、合法性和输出路径。2026 年 9 月 6 日按上述命令执行的 10 任务试运行选择 `residual_lns`，`makespan=0.390745507260`、`ratio_to_heft=0.858350393040`、`schedule_seconds=1.298581` 且 `valid=True`，完整调度保存在 `evaluation/results/schedule_output.json`。该数字用于接口示例，不纳入正式均值；墙钟时间会随机器负载变化。

输出 JSON 按 `method`、`selected_source`、`dag`、`result`、`timing`、`memory`、`diagnostics`、`validation` 和 `schedule` 九个顶层字段组织。实际文件由脚本填入真实数值；附录 E 给出各字段的类型与语义，不用容易被误读为实验结论的占位零值展示结构。

7.7 国产操作系统 Docker 矩阵

三套 Dockerfile 分别基于社区镜像：`hub.oepkgs.net/openeuler/openeuler:24.03-lts-sp4`、`quay.io/openkylin/openkylin:2.0` 和 `registry.openanolis.cn/openanolis/anolisos:23`。镜像固定 CPU-only PyTorch，并通过 `.dockerignore` 排除虚拟环境、Git 历史、文档、视频和大结果目录，降低构建上下文和身份泄露风险。

```
[powershell]
```

```
.\.venv\Scripts\python.exe scripts\run_os_matrix.py --pull --strict
```

--strict 要求每个镜像构建退出码和容器运行退出码均为 0。容器内统一执行 /etc/os-release 身份记录、Python/PyTorch/CUDA 检查、compileall、pytest 和结构场景生成。evaluation/results/os_matrix.json 同时记录输入 Dockerfile 与 requirements.txt 的 SHA-256，使后续核查者能判断证据对应哪一版环境定义。

需要强调：在 Windows 上使用 Docker Desktop 时，容器用户态来自 openEuler/openKylin/Anolis OS，但内核字段会显示 WSL2 共享内核。该结果足以证明 Python 项目在三种国产发行版用户态依赖栈中可安装、编译和测试，不等于物理机原生内核基准。若决赛环境允许，后续可在 openEuler 物理机或虚拟机复跑同一命令，证据格式无需改变。

7.8 项目目录与数据流

```
[text]
project/
├─ env/                DAG、资源、Gymnasium 环境、归一化、物理规则
├─ baselines/          HEFT、MILP、Hybrid
├─ policies/           RL 调度器、Residual Policy、重定位与 LNS
├─ training/           MaskablePPO 训练、配置、checkpoint、日志
├─ evaluation/         场景、评测、统计、调度合法性与结果 JSON
├─ diagnostics/        数据、mask、梯度、文档数字与匿名化审计
├─ docker/             三种国产操作系统 Dockerfile 与说明
├─ scripts/            复现、单场景调度、OS 矩阵、文档生成
├─ tests/              单元、性质、契约和集成测试
├─ configs/            默认云边端资源配置
├─ docs/               完整报告、图表、研究记录和过程证据
├─ scheduler_interface.py
├─ requirements.txt
└─ README.md
```

主数据流为：场景 JSON 与资源 YAML → 环境/基线加载 → Residual 或启发式构造任务顺序 → EFT 生成资源时间线 → 重定位/LNS 反复重放 → 统一合法性校验 → JSON 结果与统计汇总。训练流和推理流共用环境、物理规则及策略接口；文档生成只读取已有结果，不回写实验数据。

7.9 依赖安装与无 GPU 运行

推荐使用 Python 3.10~3.12 虚拟环境。项目正式 Docker 证据分别覆盖 3.10.12、3.11.6、3.12.2。由于调度网络规模较小且主要耗时来自 Python 搜索，默认安装 CPU-only PyTorch，torch.cuda.is_available() 为 False 是预期状态而非错误。

```
[bash]
python -m venv .venv
# Windows: .venv\Scripts\python -m pip install -r requirements.txt
# Linux:   .venv/bin/python -m pip install -r requirements.txt
python scripts/reproduce.py --profile smoke --write-manifest
```

训练 200000 步和正式 5 次×20 场景评测耗时明显高于 smoke。首次验收应先执行 smoke，确认依赖、测试、checkpoint 和场景格式正常，再运行 final。评审现场演示宜使用单场景 CLI 或 wide 小参数，不在有限展示时间内从零训练完整模型；完整训练能力由脚本与配置证明，预训练 checkpoint 用于可重复推理。

7.10 常见问题定位

表 20 常见问题与定位原则

现象	优先检查	处理原则
ModuleNotFoundError	当前解释器是否为项目虚拟环境、requirements 是否完整	用 python -m pip 确保 pip 与解释器一致
checkpoint 加载失败	.zip 是否存在、配置容量是否为 30、SB3 版本是否匹配	不静默重训或换模型，先核对版本和哈希
40~60 任务显示跳过 Residual	residual_skipped_reason	这是容量安全设计，继续看 heft_lns 输出
ratio 大于 1	是否启用 Adaptive、HEFT 是否使用同一资源配置	正式基准会断言失败，不应手工删除该场景
Docker 拉取失败	Docker daemon、网络、社区镜像仓库	重试构建；不得把失败矩阵改写成 passed
pytest 出现 17 条 warning	PuLP 弃用提示	当前测试可通过；升级 PuLP/CBC 前单独做回归

现象	优先检查	处理原则
Word 目录页码未刷新	是否由 Word COM 生成、域 是否更新	打开后全选并更新域，或重 跑文档生成脚本

第八章 局限性、适用边界与未来工作

8.1 已知局限

1. 学习模型本身仍有规模边界。支持 60 任务的历史新模型在大规模集上为 1.454469；最终系统通过容量检查、HEFT 锚点和 LNS 在 30~60 任务达到 0.941627，但 40~60 任务明确跳过 Residual 模型。因此可以声称完整调度系统安全扩展，不能声称神经网络本身已经学会任意规模泛化。
2. 宽并行 DAG 是已修复但需继续展示的历史短板。原始 Residual 模型在历史宽并行组为 1.011915，仅 2/5 场景反超 HEFT；证据见 `evaluation/results/structural_generalization/summary_structural_generalization.json`。决赛优化版通过自适应候选集将正式宽并行 64/64 结果降至 0.929848，5/5 不劣于 HEFT；但该收益来自更强的运行时组合搜索，未来仍应训练专门的宽并行鲁棒 checkpoint。
3. 推理时间换质量。Best-of-64 与 64 轮 LNS 不是实时毫秒级决策。系统已提供 fast、balanced、quality 三档及软墙钟时间预算；30~60 任务固定工作量测试的平均耗时为 6.633 秒、P95 为 13.284 秒，适合秒级离线批调度，不适用于硬实时毫秒级决策。
4. best-only 的改进边界。LNS 不会劣化当前解，但也可能停在邻域局部最优；显著性结果只适用于当前场景分布和预算。
5. MILP 覆盖有限。只有 7/10 个小场景证明最优，且 MILP 比较使用较早的 Best-of-16，而非最终 LNS。
6. 训练统计仍可加强。最终 checkpoint 来自单训练种子；推理阶段做了严格重复统计，但未来仍应增加多训练种子和置信区间。

8.2 后续方向

后续工作优先考虑：完成结构混合配置下的多训练种子鲁棒 checkpoint；探索可扩展的图指针或集合编码器；把求解器下界用于训练与停止条件；在真实 openEuler 资源遥测下引入动态负载、故障与迁移开销。任何新方法仍应遵循“先固定评测协议，再看结果”的原则。

8.3 内部有效性威胁

内部有效性关注“观察到的改善是否真的来自方法”。首先，Best-of-N 与 LNS 都增加计算量，可能把收益错误归因于算法结构；项目通过耗时略高的 Best-of-128 纯采样对照缓解这一混淆，但硬件调度噪声仍可能影响秒级测量。其次，最终 checkpoint 只有一个训练种子，当前五次重复主要覆盖推理随机性而非训练不确定性。再次，Adaptive 候选组合在发现宽并行短板后设计，宽并行专项集不能当作完全盲测；因此报告同时保留原始结构泛化结果，并把后续多训练种子和新的封存测试集列为优先事项。

8.4 外部有效性威胁

验证集规模、边密度和默认四资源配置不能代表所有云边端部署。真实系统可能存在 CPU/GPU/NPU 多维算力、非对称网络、突发负载、共享链路、抢占、失败和数据驻留限制。大规模实验扩展到 60 任务，仍不足以证明数百或数千任务上的复杂度和内存表现。容器测试覆盖三种发行版用户态，但宿主为 WSL2；未测试 ARM、RISC-V 或 OpenHarmony 原生设备。

因此本文把算法定位为“可训练、可插拔、可审计的静态 DAG 调度框架及一个已验证的高质量策略实现”，不是完整生产编排平台。面向真实部署时，应把监控、故障恢复、安全隔离和执行器接入作为独立工程层，并重新标定任务时长与通信模型。

8.5 构念与统计有效性

mean_ratio 只衡量相对 HEFT 的 makespan，不反映能耗、成本、公平性、deadline 违约率和尾延迟。HEFT 是强基线，但不是每个场景的全局最优；项目仅在小规模 MILP 已证明场景中讨论理论最优差距。五次重复的 t 检验样本量较小，正态性难以充分检验，因此 p 值与全部配对差值、场景胜率共同呈现。

工程耗时在一台本机进程中测得，RSS 包含 Python、PyTorch 和已加载模块，不等同于算法的最小独占内存。软时间预算可能略超设定值，因为原子候选不会中途终止。文档中的所有效率数字均附协议和硬件相关性说明，不把它们写成跨机器服务等级承诺。

8.6 面向实际系统的部署方式

在离线批处理场景中，上游 workflow 系统可把任务依赖、估计计算量 and 数据量导出为 node-link JSON，通过 `scripts/schedule.py` 生成完整计划，再由执行器按开始条件提交到云、边、端节点。若调度窗口严格，可选择 `fast` 或设置时间预算；若离线规划更重质量，可选择 `quality`。

在滚动重调度场景中，可定期把已完成任务从 DAG 移除，把当前资源可用时间和数据位置映射到新的资源配置，再对剩余 DAG 重新运行调度器。当前代码尚未内置这一闭环，但统一接口、JSON 输出和无副作用资源副本为接入控制器提供了基础。

在竞赛演示中，建议用三层证据说明工程可信度：先显示 `/etc/os-release` 与 Python/PyTorch，证明在 openEuler 容器中真实运行；再执行单场景 CLI，展示候选来源、ratio 和 `valid=True`；最后打开输出 JSON，展示每个任务的资源与时间段。这样可以把“算法结果”与“可运行系统”同时呈现。

8.7 决赛前建议路线图

1. 训练至少 3 个结构混合 Residual checkpoint，用封存的常规、宽并行、深链条和异构强度测试集比较均值与方差，只在稳定优于现模型时替换正式 checkpoint。
2. 给 LNS 增加增量 makespan 评估或关键路径引导，降低重复全序列重放成本，并重新测量 30~100 任务耗时曲线。
3. 在 openEuler 虚拟机或物理机运行 final profile，记录内核、CPU、内存、耗时和结果文件哈希，补强非 WSL2 证据。
4. 引入动态带宽、节点故障和任务到达的独立扩展实验，但不改变当前静态基准口径。
5. 在最终提交前同步 README、项目说明书、PPT、视频中的所有数字，运行匿名化扫描和干净克隆复现，避免材料版本不一致。

第九章 创新点与工程价值

9.1 方法创新

第一，提出 HEFT upward rank 与零初始化神经残差的直接 logit 融合。它让训练从可解释启发式出发，学习任务是“修正基线”而非“从零发现全部调度规则”，并可通过 `delta_scale` 控制学习影响。

第二，构建“随机 Residual 多样化初解—合法单任务重定位—best-only LNS”的分层搜索。学习策略提供实例相关、多样的拓扑序，约束搜索负责确定性精修，二者通过同一 EFT 重放目标衔接。

第三，引入带 HEFT anchor 的自适应候选组合。最终结果按统一 makespan 选优，使完整系统在同一调度物理规则下具有不劣于 HEFT 的设计保证；宽并行图再根据形状指标启用宽度感知和成本优先候选。

第四，把模型容量边界显式纳入调度器。超出 30 任务时不冒险调用神经网络，而是记录原因并降级到 HEFT+LNS；这一点把“算法论文中的固定张量”转化为工程系统可观察、可测试的行为。

9.2 工程创新

项目建立了从输入到证据的完整可审计链：统一 Scheduler 接口、Gymnasium 环境、MaskablePPO 训练、配置化资源、固定场景、合法性检查、重复统计、MILP 理论参照、CLI JSON 输出、三档预算、Docker OS 矩阵以及文档数字审计。任何关键结论都可以从报告跳转到原始 JSON，再定位到产生该字段的代码。

跨平台层没有只给 Dockerfile，而是保存三套镜像的构建/运行退出码、系统身份、解释器与 PyTorch 版本、测试输出、输入文件哈希。复现脚本还主动清理绝对用户路径，使可运行性证据与匿名化要求兼容。

9.3 实际价值

对于任务依赖稳定、可提前估计执行代价的离线工作流，本方法能在 HEFT 的毫秒级快速解基础上用数秒搜索换取进一步 makespan 改善。典型对象包括云端批处理

DAG、边缘推理流水线、端侧数据预处理与云侧汇总、科学工作流和构建/测试流水线。

更重要的是，框架允许替换资源配置、场景生成器和策略，而无需改写评测器。即使未来更换为图指针网络、行为克隆或新的元启发式方法，也能沿用同一合法性、基线、统计与跨平台验证体系，减少“算法换了、实验口径也跟着变”的风险。

第十章 结论

项目的核心贡献不只是得到一个小于 1 的 `mean_ratio`，而是建立了从调度物理一致性、训练诊断、动作空间设计到统计验证的完整工程闭环。最终方案让 HEFT 提供安全排序基准，让 Residual Policy 学习实例相关修正，让 EFT 保证资源放置规则一致，再用 best-only 合法重定位与 LNS 精修高质量初解。

在 5 次系统熵配对评测中，决赛优化版最终方案达到 0.916441 ± 0.000984 ，5 轮均为 20/20 场景反超 HEFT；计算量对齐的 Best-of-128 仍显著更差。与此同时，报告明确保留历史大规模模型和原始宽并行场景的负面结果，并分别给出宽并行修复及 30~60 任务容量安全扩展证据。由此得到的结论是有限但可验证的：在 8 至 25 任务及若干结构扰动场景上，Residual Best-of-64 加自适应候选集与约束搜索能够稳定优于 HEFT；在 30~60 任务上，完整系统通过显式容量检查与 HEFT/LNS 安全路径达到 0.941627、20/20 反超，但这不等价于神经网络自身完成规模外泛化，也不自动覆盖动态负载或硬实时调度。

参考文献

- [1] Topcuoglu H., Hariri S., Wu M.-Y. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260-274, 2002. DOI: <https://doi.org/10.1109/71.993206>.
- [2] Jeon W., Gagrani M., Bartan B., Zeng W. W., Teague H., Zappi P., Lott C. Neural DAG Scheduling via One-Shot Priority Sampling. *ICLR 2023*. <https://openreview.net/forum?id=WL8FlAugqQ>.
- [3] Ho K.-H., Jheng R.-Y., Wu J.-H., Chiang F., Chen Y.-C., Wu Y.-Y., Wu I.-C. Residual Scheduling: A New Reinforcement Learning Approach to Solving Job Shop Scheduling Problem. *arXiv:2309.15517*, 2023. <https://arxiv.org/abs/2309.15517>.
- [4] Ropke S., Pisinger D. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40(4):455-472, 2006. DOI: <https://doi.org/10.1287/trsc.1050.0135>.
- [5] Hottung A., Kwon Y.-D., Tierney K. Efficient Active Search for Combinatorial Optimization Problems. *ICLR 2022*. <https://openreview.net/forum?id=nO5caZwFwYu>.
- [6] Pineau J., Vincent-Lamarre P., Sinha K., Lariviere V., Beygelzimer A., d'Alche-Buc F., Fox E., Larochelle H. Improving Reproducibility in Machine Learning Research: A Report from the NeurIPS 2019 Reproducibility Program. *Journal of Machine Learning Research*, 22(164):1-20, 2021. <https://www.jmlr.org/papers/v22/20-303.html>.
- [7] 清华大学研究生院. 研究生学位论文写作指南. 2024. <https://www.dhs.tsinghua.edu.cn/wp-content/uploads/2023/12/2024031107044595.pdf>.
- [8] 中国研究生创新实践系列大赛管理平台. 第三届中国研究生操作系统开源创新大赛暨开放原子大赛操作系统专项赛: 第 16 题 “基于深度强化学习的云-边-端异构计算资源管理调度方法”. 2026. <https://cpipc.acge.org.cn/cw/hp/2c9080178c7c917b018d1b1a0af61cd6>.
- [9] 中国研究生创新实践系列大赛管理平台. 第三届中国研究生操作系统开源创新大赛暨开放原子大赛操作系统专项赛初赛作品提交说明. 2026-05-23. <https://cpipc.acge.org.cn/cw/contestNews/detail/2c9080178c7c917b018d1b1a0af61cd6/2c9080179e4a7a92019e52efe7320ddf?page=1>.

- [10] Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>.
- [11] Raffin A. et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. Journal of Machine Learning Research, 22(268):1-8, 2021. <https://www.jmlr.org/papers/v22/20-1364.html>.
- [12] Stable Baselines3 Contrib Developers. Maskable PPO Documentation. https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html.
- [13] Towers M. et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032, 2024. <https://arxiv.org/abs/2407.17032>.
- [14] PyTorch Contributors. Reproducibility: Randomness in PyTorch. <https://docs.pytorch.org/docs/stable/notes/randomness.html>.
- [15] openEuler Community. openEuler Community Releases. <https://www.openeuler.org/en/download/>.
- [16] openKylin Community. openKylin Official Website. <https://www.openkylin.top/>.
- [17] OpenAnolis Community. OpenAnolis Official Website. <https://openanolis.cn/>.

附录 A 关键证据索引

表 21 关键结论与原始证据索引

结论	原始证据
决赛优化版最终五次统计	evaluation/results/final_pipeline_lns_summary.json
初赛 LNS 五次统计	evaluation/results/autonomous_exploration/direction2_lns/direct_vs_residual_paired_summary.json
计算量对齐 Best-of-128	evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json
自主探索时间线与预注册式决策	docs/自主探索日志.md
MILP 精确解与界	evaluation/results/milp_optimal_comparison.json
结构化泛化历史短板	evaluation/results/structural_generalization/summary_structural_generalization.json
宽并行专项修复	evaluation/results/wide_parallel_adaptive.json
大规模负面结果	evaluation/results/summary_mlp_normalized_large.json
30~60 任务 10 秒预算结果	evaluation/results/final_adaptive_large_scale.json
30~60 任务固定工作量结果	evaluation/results/final_adaptive_large_scale_fixed.json
单场景 CLI 与合法性输出	scripts/schedule.py、evaluation/schedule_validation.py
openEuler 环境与测试	evaluation/results/openeuler_validation/
openEuler/openKylin/Anolis Docker 矩阵入口与本机诊断	docker/、evaluation/results/os_matrix.json
全新克隆最终 LNS 一键流水线	evaluation/results/run_final_pipeline_lns_fresh_clone_full_output.log

附录 B 关键代码索引

表 22 关键代码索引

功能	文件
插入式调度规则	env/scheduling_utils.py
Joint / Ranked / Residual 环境	env/scheduling_env.py、 env/scheduling_env_ranked.py、 env/scheduling_env_residual.py
HEFT upward rank	baselines/heft_scheduler.py
Residual logits	policies/residual_ranking_policy.py
Best-of-N	policies/rl_scheduler.py
合法重定位与 LNS	policies/residual_local_search_scheduler.py
MaskablePPO 训练	training/train_ppo.py
最终 LNS 评测	evaluation/evaluate_residual_lns.py
配对统计	evaluation/analyze_residual_lns_direct_repeats.py
结构形状与 Adaptive portfolio	policies/residual_local_search_scheduler.py
大规模耗时/内存基准	evaluation/benchmark_scalability.py、 evaluation/runtime_metrics.py
最终调度合法性检查	evaluation/schedule_validation.py
单场景机器可读入口	scripts/schedule.py
三发行版严格矩阵	scripts/run_os_matrix.py、docker/
报告审计	diagnostics/technical_report_result_audit.py

附录 C 匿名化与数据口径声明

本报告封面不填写学校、指导教师和队员姓名。Word 核心属性中的作者和最后保存者统一写为 **Anonymous**。报告只采用仓库现存原始结果；历史描述与文件冲突时以 JSON/日志为准。单次固定种子结果只用于复现示例，最终结论使用重复评测均值与样本标准差。未证明最优的 MILP 可行解不称为全局最优。所有绝对路径、Windows 用户目录和本机身份信息均不得写入正式报告正文或证据日志。

附录 D 最终训练与推理配置

表 23 最终训练与推理参数

参数	最终值	作用
训练任务数	8~25	常规训练分布
训练边密度	0.2~0.5	控制 DAG 稀疏程度
max_tasks_padding	30	模型固定输入容量
训练步数	200000	MaskablePPO 总环境步数
学习率	0.0003	PPO 优化器步长
n_steps	2048	每轮 rollout 步数
batch size	64	小批量大小
epoch 数	10	每轮 rollout 的优化遍数
gamma	0.99	折扣因子
gae_lambda	0.95	GAE 参数
clip_range	0.2	PPO 裁剪范围
熵系数	0.01	保持探索
MLP 隐层	[128, 128]	actor/critic 主干
residual delta scale	1.0	神经残差权重
HEFT rank scale	1.0	启发式先验权重
训练种子	123	checkpoint 训练随机源
quality 推理	64/3/64	采样/重定位/LNS
balanced 推理	32/2/32	默认质量一耗时折中
fast 推理	8/1/8	现场或低延迟使用

最终权威配置是 training/configs/ppo_mlp_residual.yaml。若本表与 YAML 不一致，应以 YAML 和 checkpoint 实际输入空间为准，并重新生成说明书；不允许只改文档数字而不改代码证据。

附录 E 单场景输入输出字段说明

输入 DAG 使用 NetworkX node-link JSON。节点至少需要可转换为整数的任务编号和 computation；边需要 source、target 与 data_size。资源 YAML 每项包含 id、tier、compute_power 和 bandwidth。输入校验会拒绝空图、有环图、缺少计算量、非有限数值和未知资源。

表 24 单场景调度输出字段

输出字段	类型	语义
method	string	adaptive 或 heft
selected_source	string	最终胜出的候选来源
dag.task_count/edge_count	integer	输入图规模
result.makespan	number	最晚任务完成时间
result.heft_makespan	number	同场景 HEFT 参考
result.ratio_to_heft	number	主比较指标
timing.model_load_seconds	number	checkpoint 与策略加载时间
timing.schedule_seconds	number	调度阶段墙钟时间
memory.process_peak_rss_mib	number/null	当前进程峰值常驻内存
diagnostics.candidate_makespans	object	每个候选的 makespan
diagnostics.time_limit_reached	boolean	是否触及软截止
diagnostics.residual_skipped_reason	string/null	模型未参与的明确原因
validation.valid	boolean	完整合法性检查是否通过
validation.errors	array	若失败，列出具体约束错误
schedule[]	array	每个任务的资源、开始和完成时间

附录 F 自动化测试覆盖矩阵

表 25 自动化测试覆盖矩阵

测试文件	覆盖重点	防止的回归
tests/test_env.py	DAG 无环、 reset/step、HEFT 合法、 mask	基础环境与合法动作 损坏
tests/test_scheduling_consistency.py	同分配下环境与 HEFT makespan 一致	物理规则不公平
tests/test_scheduling_env_ranked.py	Ranked 动作和 HEFT 顺序重放	槽位/任务编号错配
tests/test_reward_shaping.py	相对 HEFT 奖励	参考值污染或符号错 误
tests/test_residual_scheduling.py	零初始化 residual、 HEFT 等价起点	训练起点偏离基线
tests/test_residual_local_search_scheduler.py	重定位/LNS 拓扑合法 且不退化	搜索产生非法或更差 解
tests/test_portfolio_robustness.py	宽图、截止、容量降 级、HEFT anchor	宽并行和越界崩溃
tests/test_schedule_cli.py	JSON 可序列化、合法 性接受/拒绝	实际接口只跑不验
tests/test_milp_optimal_scheduler.py	已知最优场景和 bound 解析	理论参照错误
tests/test_hybrid_scheduler.py	MILP 成功/失败回退	插拔调度器分支错误
tests/test_os_matrix_config.py	三镜像、CPU-only、 仓库与隐私	跨平台配置漂移
tests/test_structural_generalization.py	结构分组与固定路径	泛化场景被意外覆盖
tests/test_rl_scheduler_interface.py	RL 统一接口	评测器无法调用策略
test_gat_*	图编码器形状与加载 smoke	历史可插拔方法失效

附录 G 关键文件完整性指纹

以下 SHA-256 在 2026 年 9 月 5 日由当前最终代码工作区读取，用于识别本报告引用的证据版本。重新运行实验或更新依赖后哈希变化是正常现象，但应同步更新说明书和审计记录。

表 26 关键证据文件完整性指纹

文件	大小/字节	SHA-256
training/checkpoints/ppo_mlp_residual.zip	403 5080	2f7593f4f250b775fa6a5b89d2dfc212ed00f0fc1003e602b8ea7a54e339da4d
evaluation/results/final_pipeline_ins_summary.json	339 4	74d40d011c6d4a264ca518fa63cbb2018c771e7e8cb e05749a965c9c401a9bae
evaluation/results/wide_parallel_adaptive.json	904 2	52cfe7c8542b06a5659acee9c895ae8b44bd3c12dbe 932ffb61dbdda2c822214
evaluation/results/final_adaptive_lar_ge_scale.json	408 84	0b7bb870033cd4dab01937915380f26e5ee5ff23aeb 649fb192d6f78276d8d5b
evaluation/results/final_adaptive_lar_ge_scale_fixed.json	418 06	5874fcf112494f88de796951c1fb9b626a6c959c4e7 66d6ebe88b39b5e05cc29
evaluation/results/os_matrix.json	136 54	32da47acbfcdacffd8b2e28cff02437dd7476a77bdc bfe2ef0222cf854fae20
requirements.txt	378	08cbc03e1737759cf7180bfceac88019f77b1912027f619ff86cdc136cd6aeeb

附录 H 评审快速核验路径

如果只有十分钟核查仓库，建议按以下顺序：

1. 打开根目录 README，确认项目定位、主结果和运行入口。
2. 查看 `training/configs/ppo_mlp_residual.yaml`，确认任务规模、训练参数和 `checkpoint` 路径。
3. 查看 `evaluation/results/final_pipeline_ins_summary.json`，核对 0.916441 ± 0.000984 、五次种子和 `p` 值。
4. 查看 `policies/residual_local_search_scheduler.py`，确认 HEFT anchor、Residual 容量检查、best-only 接受和 Adaptive 候选选优。
5. 运行 `python scripts/reproduce.py --profile smoke --write-manifest`，确认编译、测试和场景生成。
6. 运行 `scripts/schedule.py` 示例，确认输出完整调度且 `validation.valid=true`。
7. 查看 `evaluation/results/os_matrix.json`，核对三种发行版身份、构建/运行退出码和容器测试输出。
8. 查看本报告第八章，确认项目没有隐瞒模型容量、动态负载和 WSL2 边界。

附录 I 术语表

表 27 主要技术术语

术语	含义
DAG	有向无环图，用节点表示任务、边表示依赖与数据传输
makespan	从计划开始到最后一个任务完成的总时间
HEFT	面向异构计算的列表调度启发式，使用 upward rank 与 EFT
EFT	Earliest Finish Time，给定任务在候选资源上的最早完成时间
DRL	Deep Reinforcement Learning，深度强化学习
MaskablePPO	在 PPO 策略分布中支持非法动作屏蔽的实现
Residual Policy	在 HEFT rank 基础上学习有界修正的策略
Best-of-N	对同一实例随机生成 N 个完整候选并选最优
LNS	Large Neighborhood Search，破坏并修复部分解的大邻域搜索
best-only	仅接受严格改善候选的策略
portfolio	同时构造多种来源候选并按统一目标选优
action mask	标识当前状态下哪些离散动作合法的布尔向量
padding capacity	神经网络固定输入张量允许的最大任务数
RSS	Resident Set Size，进程当前驻留物理内存
paired test	在同一场景与种子上比较两方法差值的统计检验